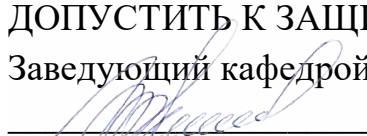


Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Уральский федеральный университет
имени первого Президента России Б. Н. Ельцина»
Институт радиоэлектроники и информационных технологий-РТФ
Кафедра информационных технологий и систем управления

ДОПУСТИТЬ К ЗАЩИТЕ ПЕРЕД ГЭК
Заведующий кафедрой ИТиСУ
 Е. В. Кислицын
«30» мая 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

**ИССЛЕДОВАНИЕ МЕТОДОВ ДЕТЕКТИРОВАНИЯ ИЗОБРАЖЕНИЙ,
СГЕНЕРИРОВАННЫХ ДИФфуЗИОННЫМИ МОДЕЛЯМИ**

Научный руководитель: Корелин Иван Андреевич
к.т.н., доцент

Нормоконтролер: Огуренко Егор Владимирович

Студент группы РИМ-231902 Горбачев Алексей Николаевич



подпись

подпись



подпись

Екатеринбург
2025

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Уральский федеральный университет имени первого Президента России Б.Н. Ельцина»

Институт радиоэлектроники и информационных технологий – РТФ
Кафедра информационных технологий и систем управления
Направление подготовки 09.04.01 Информатика и вычислительная техника
Образовательная программа 09.04.01/33.03 Инженерия машинного обучения

ЗАДАНИЕ
на выполнение магистерской диссертации

студента Горбачева Алексея Николаевича группы РИМ-231902
(фамилия, имя, отчество)

1. Тема выпускной квалификационной работы

Исследование методов детектирования изображений, сгенерированных диффузионными моделями

Утверждена распоряжением по институту от «2» декабря 2024 г. № 33.02-05/334

2. Научный руководитель

Корелин Иван Андреевич, к.т.н., доцента кафедры информационных технологий и систем управления ИРИТ-РТФ

3. Исходные данные к работе

Нормативная, учебная, методическая литература по теме магистерской диссертации, материалы, полученные в ходе преддипломной практики, научная статья

4. Перечень демонстрационных материалов

Презентация, архитектура программного комплекса

5. Календарный план

№ п/п	Наименование этапов выполнения работы	Срок выполнения этапов работы	Отметка о выполнении
1.	1 раздел (глава)	до 24.03.2025 г.	Выполнено
2.	2 раздел (глава)	до 28.04.2025 г.	Выполнено
3.	3 раздел (глава)	до 19.05.2025 г.	Выполнено
4.	ВКР в целом	до 23.05.2025 г.	Выполнено

Руководитель _____
Андреевич
(подпись)

Корелин Иван
(Ф.И.О.)

Студент задание принял к исполнению 10.02.2025 г.
дата

(подпись)

6. Допустить Горбачева Алексея Николаевича к защите магистерской диссертации в Государственной экзаменационной комиссии.

Заведующий кафедрой ИТиСУ
(подпись)

Е. В. Кислицын
(Ф.И.О.)

РЕФЕРАТ

Магистерская диссертация 80 с., 26 рис., 11 табл., 50 источн., 2 прил.
ИССЛЕДОВАНИЕ МЕТОДОВ ДЕТЕКТИРОВАНИЯ ИЗОБРАЖЕНИЙ,
СГЕНЕРИРОВАННЫХ ДИФФУЗИОННЫМИ МОДЕЛЯМИ.

Объект исследования – методы выявления сгенерированных изображений.

Предметом исследования является архитектура, позволяющая детектировать изображения, сгенерированные диффузионными моделями.

Цель данной работы – исследование архитектур и алгоритмов, позволяющие детектировать изображения, сгенерированные с помощью диффузионных моделей, и разработка более эффективного алгоритма детектирования.

Методы исследования: изучение существующих методов детектирования сгенерированных изображений, генерации идей, анализ и синтез, алгоритмические эксперименты.

Результатом работы является описание архитектуры и алгоритма детектора сгенерированных изображений.

Выпускная квалификационная работа выполнена в текстовом редакторе и представлена в электронном виде.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	4
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	6
ВВЕДЕНИЕ	7
Глава 1. Обзор существующих проблем и методов решения.	10
1.1. Проблемы детектирования сгенерированных изображений ...	10
1.2. Анализ существующих моделей генерации изображений	11
1.3. Анализ существующих методов детектирования сгенерированных изображений	14
1.4. Выводы по главе 1	29
Глава 2. Исследование и анализ выбранной архитектуры и методологии	31
2.1. Описание проводимого эксперимента.....	31
2.2. Описание и подготовка набора данных.....	31
2.3. Описание выбранной модели для проведения исследования .	35
2.4. Обучение модели и методы оценки	39
2.5. Выводы по главе 2	46
Глава 3. Анализ работы и результаты проведения эксперимента	48
3.1. Экспериментальная инфраструктура и условия тестирования	48
3.2. Сравнительный анализ метрик классификации.	50
3.3. Интерпретация результатов	53
3.4. Визуализация результатов.....	65
3.5. Выводы по главе 3	72
ЗАКЛЮЧЕНИЕ	74

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	76
ПРИЛОЖЕНИЕ А	81
ПРИЛОЖЕНИЕ Б.....	90

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

ADM – адаптивная диффузионная модель удаления шума (Adaptative Denoising Model)

BLEU – Bilingual Evaluation Understudy Score – метрика для оценки качества машинного перевода, измеряющая точность через совпадение n–грамм с эталонными переводами

BLIP – модель представления текстового описания изображения (Bootstrapping Language–Image Pre–training)

CIDEr – Consensus–based Image Description Evaluation – метрика для оценки описания изображений, измеряющая согласованность с множеством эталонных описаний, учитывая частотность и значимость n–грамм.

CLIP – модель представления текстовых эмбеддингов описания изображения (Contrastive Language–Image Pre–Training)

CNN – сверточная нейронная сеть

DCT – дискретное косинусное преобразование

DFT – дискретное преобразование Фурье

DM – диффузионная модель (diffusion model)

FFT – быстрое преобразование Фурье

GAN – генеративно–сопоставительные сети (generative adversarial network)

GLIDE – модель, объединяющая диффузионные методы и язык для генерации и редактирования изображений из текстовых описаний. (Guided Language–to–Image Diffusion for Generation and Editing)

IFFT – обратное быстрое преобразование Фурье

ROUGE – Recall–Oriented Understudy for Gisting Evaluation – метрика для оценки автоматического суммирования текста, сравнивающая совпадение n–грамм, предложений и лексем

VAE – вариационный автокодировщик (variational autoencoder)

VQDM – диффузионная модель с векторным квантованием (Vector Quantized Diffusion Models)

ВВЕДЕНИЕ

Быстрое развитие диффузионных моделей значительно улучшило качество генерации изображений, в результате чего созданный контент все труднее отличить от реальных изображений. Их широкое распространение также порождает новые вызовы, связанные с распространением дезинформации, фейковых новостей и манипуляций с визуальным контентом. В связи с этим возникает острая необходимость в разработке эффективных методов детектирования изображений, сгенерированных диффузионными моделями, чтобы обеспечить доверие к цифровому контенту и предотвратить злоупотребления.

Однако обнаружению изображений, созданных с помощью диффузионных моделей, уделяется относительно мало внимания, что вызывает опасения по поводу потенциального злоупотребления ими в злонамеренных целях и крайне важно для предотвращения негативного воздействия на наше общество. В отличие от этого, генеративные состязательные сети (GAN) широко изучались с криминалистической точки зрения. Поэтому в этом проекте делается упор именно на диффузионные модели.

В данном проекте исследуется создание детектора, позволяющего отличать реальные изображения от изображений, сгенерированных диффузионными моделями. Было обнаружено, что существующие детекторы с трудом распознают изображения, сгенерированные с помощью диффузионных моделей, даже если включаются в их обучающие данные изображения, сгенерированные с помощью конкретной диффузионной модели.

Гипотеза исследования предполагает, что изменение архитектуры FIRE [1] позволит улучшить качество выявления сгенерированных изображений.

Актуальность проблемы заключается в следующих факторах:

1) увеличение доступности и качества генеративных моделей приводит к массовому созданию синтетического контента, что требует разработки инструментов для его идентификации;

2) угрозы информационной безопасности: фейковые изображения могут использоваться для манипуляции общественным мнением, мошенничества и других злонамеренных целей;

3) традиционные подходы к детектированию поддельных изображений, такие как анализ артефактов сжатия или статистических аномалий, часто оказываются неэффективными против современных диффузионных моделей, которые создают изображения с высокой степенью реалистичности;

4) этические и правовые аспекты: использование технологии «дипфейк» при совершении преступления может стать отягчающим обстоятельством, поэтому Глава Минцифры предложил создать в России приложения, которые будут распознавать «дипфейки» в режиме реального времени [2].

Цель данной работы – исследование архитектур и алгоритмов, позволяющие детектировать изображения, сгенерированные с помощью диффузионных моделей, и разработка более эффективного алгоритма детектирования. Исходя из поставленной цели в настоящей магистерской работе реализуются следующие задачи:

- 1) рассмотреть текущие методы детектирования изображений,
- 2) сформировать архитектуру и алгоритм для детектирования изображений,
- 3) подготовить набор данных для обучения и оценки качества алгоритма,
- 4) провести эксперимент по обучению детектора.

Объектом исследования являются методы выявления сгенерированных изображений.

Предметом исследования является архитектура, позволяющая детектировать изображения, сгенерированные диффузионными моделями.

При написании данной работы использовались следующие методы: математическое моделирование, расчетно–конструктивный, абстрактно–логический, анализ и другие методы.

Научной новизной в работе являются:

1) исследование уникальных особенностей изображений, созданных диффузионными моделями, таких как частотное распределение компонентов изображения, которые могут быть использованы для их идентификации,

2) создание новых архитектур, способных эффективно выделять и классифицировать признаки, характерные для синтетических изображений, с учетом их высокой реалистичности,

3) применение современных подходов машинного обучения для обучения моделей на больших наборах данных, включающих как реальные, так и синтетические изображения,

4) исследование возможностей использования дополнительных данных, таких как текстовые описания, для повышения точности детектирования.

Практической ценностью работы является то, что разработанный алгоритм лучше справляется с детектированием сгенерированных диффузионными моделями изображений, чем существующие методы.

Результатом магистерской работы является подробное описание и создание алгоритма детектирования сгенерированных изображений.

Данная работа состоит из введения, трех разделов, заключения и списка использованных источников, включающего 50 наименований. Работа изложена на 80 страницах машинного текста, иллюстрирована 26 рисунками и сопровождается 11 таблицами.

Глава 1. Обзор существующих проблем и методов решения.

1.1. Проблемы детектирования сгенерированных изображений

На сегодняшний день генеративные модели, такие как диффузионные модели, стали мощным инструментом для создания фотореалистичных изображений. Эти модели, использующие прогрессирующее восстановление изображений из шума [3], демонстрируют впечатляющую способность к синтезу визуального контента, который трудно отличить от реальных фотографий [4]. С одной стороны, это открывает новые возможности в различных областях, таких как компьютерная графика, медицина, визуальные эффекты и искусство. Однако столь быстрый прогресс сопровождается вопросами о безопасном использовании этих технологий, а также рисками, связанными с их потенциальными злоупотреблениями.

Ключевая проблема заключается в том, что высокое качество изображений, сгенерированных диффузионными моделями, делает их практически неотличимыми от реальных. В результате злоумышленники могут использовать такие изображения для создания поддельного контента, распространения дезинформации, компрометации личности и введения людей в заблуждение [5]. Это создает серьёзные угрозы не только для частной жизни, но и для общественной безопасности, политических институтов и экономики. В условиях роста числа фейковых изображений потребность в надёжных инструментах и алгоритмах для их детектирования усиливается.

Дополнительная сложность заключается в том, что методы, обеспечивающие генерацию изображений, непрерывно совершенствуются, что приводит к ещё большей реальности генерируемого контента. В частности, диффузионные модели превосходят другие генеративные подходы, такие как GAN (Generative Adversarial Networks), по качеству и разнообразию создаваемых изображений. Это ставит перед исследователями новые задачи: разработка более точных, надёжных и универсальных методов

детектирования, которые могли бы своевременно адаптироваться к изменениям в технологиях генерации [6].

На текущий момент существующие методы детекции изображений, таких как поиск цифровых следов генерации, анализ артефактов или использование нейронных сетей, также сталкиваются с определёнными ограничениями. С развитием диффузионных моделей такие подходы становятся менее эффективными, поскольку алгоритмы генерации избегают создания визуальных изъянов и обучаются более тщательно подстраивать сгенерированные изображения под статистические закономерности реальных данных [7]. Более того, интерес вызывает аспект борьбы инновационных решений: технологии генерации изображений и их детектирования развиваются параллельно, создавая динамичное противостояние между создателями и детекторами поддельного контента.

Исходя из вышеизложенного, актуальность изучения методов идентификации изображений, сгенерированных диффузионными моделями, становится очевидной. Данная работа направлена на анализ текущих вызовов в этой области, исследование существующих подходов к детектированию синтетического контента, а также поиск новых решений, способных эффективно противостоять возникающим угрозам. В ходе работы особое внимание будет уделено исследованию специфики операций диффузионных моделей, чтобы с большей точностью выявлять малейшие признаки искусственного синтеза.

1.2. Анализ существующих моделей генерации изображений

1.2.1. Генеративно–сопоставительные сети (GAN).

Генеративно–сопоставительные сети (Generative Adversarial Networks, GAN) – это класс алгоритмов машинного обучения, которые используются для создания новых данных на основе уже существующих.

Концепция GAN заключается в том, что есть две нейронные сети: генератор и дискриминатор. Генератор создает новые изображения из случайного шума, а дискриминатор пытается отличить сгенерированные изображения от настоящих [8]. В процессе обучения, генератор учится создавать все более реалистичные изображения, чтобы обмануть дискриминатор.

Процесс обучения представляет собой минимаксную игру с целевой функцией:

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [1 - \log D(G(z))], \quad (1)$$

где генератор G учится генерировать правдоподобные данные, а дискриминатор D совершенствуется в их распознавании.

Благодаря этому подходу, GAN позволяют получать изображения высокого качества. Одной из сложностей GAN является их тренировочная нестабильность и сложность оценки качества сгенерированных данных, что привело к разработке большого количества усовершенствований базовой архитектуры.

Безусловные GAN, такие как BigGAN и StyleGAN, изучают скрытое распределение реальных выборок и генерируют высококачественные изображения путем случайной выборки из изученного скрытого пространства.

Условные GAN могут использовать входные изображения в качестве условных ограничений для формирования сгенерированных изображений. Такие модели, как CycleGAN и StarGAN, разработаны с учетом этой цели и позволяют выполнять такие задачи, как перевод изображений и передача стиля.

Для GAN с текстовой обработкой требуется только текстовое сопровождение. запрос на указание содержания и стиля сгенерированных изображений, представленных GALIP и GigaGAN.

GAN используются для генерации изображений, стилизационного переноса, синтеза текста, повышения разрешения (Super-Resolution) и в медицинских исследованиях.

Преимуществами GAN являются высокое качество генерации, универсальность в задачах обучения без учителя. А к недостаткам можно отнести сложность стабилизации обучения и вычислительная затратность.

GAN остаются ключевым инструментом в области генеративного моделирования, сочетая теоретическую глубину с широким спектром практических приложений.

1.2.2. Диффузионные модели (DM).

Диффузионные модели представляют собой передовой тип искусственной нейронной сети, основанный на итеративном процессе добавления и удаления шума – декомпозиции процесса генерации данных (sampling) используя множество относительно небольших шагов удаления шума (denoising) [3]. В настоящее время диффузионные модели широко применяются в задачах генерации изображения [4] и звука [9], также ведутся исследования по использованию их в анализе временных рядов [10].

Концептуально, работа модели DDPM [3] описывается следующим образом: на обучающие данные итеративно накладывается гауссовский шум по заданному расписанию (линейному, косинусному и др.), гиперпараметром сети служит число шагов T . На каждом шаге t добавляется шум, превращая исходные данные в чистый шум через T итераций. Этот процесс можно записать математически следующим образом [11] (здесь ε_{t-1} – стандартное гауссово распределение с нулевым средним значением и единичной дисперсией):

$$x_t = \sqrt{1 - \beta_t} \cdot x_{t-1} + \sqrt{\beta_t} \cdot \varepsilon_{t-1}, \quad (2)$$

где β_i – расписание диффузии.

После этого сеть итеративно производит удаление шума на t ($t < T$) обучающих шагах путём предсказания шума на каждом шаге, минимизируя MSE–ошибку между предсказанным и реальным шумом, используя в качестве корректного ответа зашумлённые данные с $t + 1$ шага.

Модель DDIM [12] предлагает использовать метод детерминированных цепей Маркова для уменьшения количества этапов диффузии, что позволяет ускорить скорость генерации изображений.

Модель ADM [4] впервые превзошла модель GAN в генерации изображений за счет внедрения классификатора для повышения качества генерируемых изображений.

Такие модели как Stable Diffusions [13; 14] и Midjourney [15], являющиеся диффузионными моделями, преобразующими текст в изображение, достигли выдающихся возможностей по генерации изображений, используя процесс диффузии в латентном пространстве и обучающихся на очень большом наборе данных.

Преимуществами диффузионных моделей являются высокое качество и разнообразие генерируемых данных, устойчивость к коллапсу мод (в сравнении с GAN) и гибкость в управлении процессом через расписание и условия.

Диффузионные модели нашли применение в синтезе изображений, аудио и видео [16], демонстрируя «state-of-the-art» результаты в задачах генеративного моделирования.

1.3. Анализ существующих методов детектирования сгенерированных изображений

Отличать реальные изображения от изображений, созданных с помощью диффузионных моделей (таких как DALL·E, Stable Diffusion или MidJourney),

становится все более сложной задачей по мере совершенствования этих моделей [17]. Однако существует несколько методов и стратегий, позволяющих отличить реальные изображения от изображений, созданных искусственным интеллектом:

1. Визуальный осмотр

а) Артефакты и несовершенства: диффузионные модели иногда создают едва заметные артефакты, такие как:

- неестественные текстуры или узоры;
- непоследовательное освещение или тени;
- странности в мелких деталях (например, руках, зубах или тексте);
- размытие или чрезмерное сглаживание в определенных областях.

б) Контекстуальные ошибки: ищите нелогичные или несогласованные элементы на изображении, такие как:

- объекты, которые не соответствуют физике реального мира;
- неправильно расположенные или искаженные объекты;
- необычные пропорции или ракурсы.

2. Анализ метаданных

Метаданные изображения (например, данные EXIF) могут указывать на его происхождение:

- реальные изображения часто содержат метаданные о камере, местоположении и временной метке;
- изображения, созданные с помощью искусственного интеллекта, могут не содержать метаданных или содержать необычные записи.

Примечание: метаданные могут быть удалены или подвергнуты манипуляциям, поэтому этот метод не является надежным.

3. Форензика

Используйте инструменты форензики для выявления несоответствий:

- Шумовые характеристики: реальные изображения имеют одинаковые шумовые характеристики с датчика камеры, в то время как изображения, сгенерированные искусственным интеллектом, могут содержать неестественный шум.
- Артефакты сжатия: изображения, сгенерированные искусственным интеллектом, могут отличаться от реальных фотографий.
- Частотный анализ: такие инструменты, как преобразование Фурье, могут выявлять аномалии в частотной области.

4. Инструменты обнаружения на основе искусственного интеллекта

Возможно обучить или использовать предобученную модель машинного обучения, специально разработанную для обнаружения изображений, созданных с помощью искусственного интеллекта:

- классификаторы: такие модели, как CNN (сверточные нейронные сети), можно обучить отличать реальные изображения от сгенерированных;
- предварительно обученные инструменты: некоторые инструменты и API-интерфейсы уже доступны для обнаружения контента, сгенерированного искусственным интеллектом (например, Deepware, Sensity AI). Эти инструменты анализируют шаблоны и особенности, которые человеку трудно распознать.

5. Водяные знаки и их происхождение

Некоторые модели искусственного интеллекта встраивают едва заметные водяные знаки или подписи в созданные изображения. Поиск данных маркеров можно обеспечить с помощью специализированных инструментов.

6. Поведенческий анализ

Возможно также проанализировать контекст, в котором было опубликовано изображение:

- получено ли оно из надежного источника?
- соответствует ли оно известным событиям или местоположениям?

Изображения, созданные искусственным интеллектом, часто публикуются в контекстах, где их подлинность сомнительна.

7. Опыт человека

Привлечение экспертов в предметной области (например, фотографов, судебных аналитиков) для тщательного изучения изображения может являться одним из способов обнаружения поддельных изображений. Эксперты могут обнаружить незначительные несоответствия, которые могут быть пропущены автоматизированными инструментами.

Комбинированием этих методов, можно улучшить способность распознавать изображения, созданные с помощью искусственного интеллекта. Однако по мере дальнейшего развития искусственного интеллекта грань между реальным и сгенерированным контентом, скорее всего, станет еще более размытой.

1.3.1. Модели пространственной области поиска артефактов в сгенерированных изображениях.

Такие модели основаны на пространственной области изображений, например, такие как RGB и HSV. Некоторые подходы [18] используют специфические особенности, возникающие в процессе синтеза, такие как цветовая характеристика или характеристика формы.

В некоторых исследованиях [19] используются характеристики цветового пространства для классификации поддельных и реальных изображений.

Например, анализ уровня ошибок [20] использует ошибки на уровне пикселей для обнаружения подделки изображений. Некоторые методы [21; 22] используют глубокие нейронные сети для извлечения высокоуровневой информации из пространственной области и добились заметного прогресса. Анализ артефактов GAN [22] использует дискриминатор для выявления специфических схем манипулирования изображениями. Однако ввиду того, что они используют только информацию о пространственной области они нечувствительны к более тонким артефактам, которые трудно обнаружить в цветовом пространстве. Также, исследование артефактов в пространственной области было актуально для изображений, сгенерированных GAN-моделями, т.к. изображения, сгенерированные DM-моделями менее подвержены пространственным искажениям.

Основные модели, использующие пространственные артефакты, являются [23] и [24]. Но как видно, подход, использующий только пространственные артефакты устарел и применялся только в детектировании GAN-изображений и на текущий момент не актуален.

1.3.2. Модели частотной области поиска артефактов в сгенерированных изображениях.

Анализ частотной области является классическим и важным методом обработки изображений и широко используется в ряде приложений, таких как классификация изображений, стегоанализ, классификация текстур и сверхразрешение.

Спектр изображений, сгенерированных GAN, содержит видимые артефакты в виде периодического, похожего на сетку, узора из-за перенесенных операций свертки [25]. Эти результаты были позже воспроизведены [26] и расширены до дискретного косинусного преобразования (DCT) [27]. Другая особенность была обнаружена в работе [28], которая показала, что GAN не способны корректно воспроизвести

спектральное распределение обучающих данных. В частности, сгенерированные изображения содержат увеличенные амплитуды на высоких частотах. Хотя эти специфические расхождения объясняются увеличивающими свертками (transposed convolutions) или, в более общем плане, операциями увеличения размера (upsampling).

Напротив, для изображений, сгенерированных DM, частотные паттерны, похожие на сетку, сильно зависят от конкретной модели. Также утверждается, что диффузионные модели имеют индуктивное смещение, согласно которому во время обратного процесса более высокие частоты добавляются к существующим более низким частотам [29].

Рассмотрев средний спектр изображений после DFT, можно заметить, что для всех GAN-моделей преобладают значительные артефакты, преимущественно в виде регулярной сетки, что соответствует предыдущим результатам [25–27]. И напротив, DFT спектр изображений, сгенерированных DM, значительно больше похож на спектр реальных изображений с почти незаметными артефактами (см. Рисунок 1) [7], кроме LDM. Ввиду того, что его архитектура отличается от остальных DM тем, что изображение генерируется состязательно обученным автоэнкодером, что объясняет расхождение. Это подтверждается, что спектральный шаблон обусловлен наличием состязательного дискриминатора в GAN-моделях [30; 31]

Таким образом, можно сделать вывод, что традиционные DM, которые генерируют изображения постепенным уменьшением шума (denoising), не создают частотных артефактов, подобных для GAN-моделей. Поэтому детекторы, обученные на изображениях, сгенерированных GAN-моделями, не подходят для изображений, сгенерированных DM.

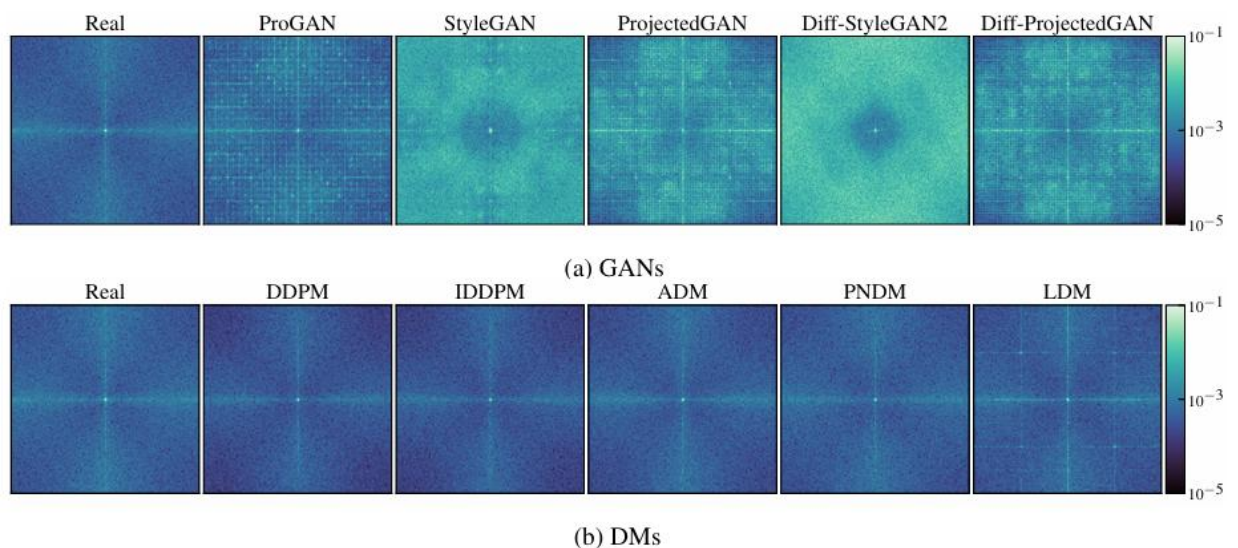


Рисунок 1 – Средний спектр DFT реальных и сгенерированных изображений

CNNDetection [32] – это метод, разработанный для обнаружения изображений, сгенерированных с помощью генеративно–состязательных сетей (GAN), таких как ProGAN, StyleGAN, BigGAN и других. CNNDetection использует сверточные нейронные сети (CNN) для анализа изображений и определения, являются ли они синтетическими или реальными.

CNN используются для анализа изображений, извлечения признаков и классификации. Они эффективны для работы с визуальными данными благодаря своей способности улавливать локальные и глобальные особенности изображений.

GAN–изображения часто содержат артефакты или несоответствия, которые не видны человеческому глазу, но могут быть обнаружены CNN. Например, это могут быть нереалистичные текстуры, аномалии в освещении или несоответствия в деталях.

CNNDetector обнаруживает артефакты в частотной области CNN–подобных моделях, позволяя детектировать сгенерированные изображения. Это первый «state-of-the-art» детектор, способный обнаруживать сгенерированные изображения через постобработку. Ввиду того, что диффузионные модели также используют CNN при генерации изображений

(имеются CNN–слои со слоем увеличения размера (upsampling) в U–net–сети диффузионной модели), то данный детектор также считался эталонным для изображений, сгенерированных диффузионными моделями.

F³–Net. Frequency in Face Forgery Network [33] – это двухпоточная нейросеть, которая анализирует частотные характеристики изображений для обнаружения подделок. Ключевая идея: артефакты подделок проявляются в частотной области, даже если они незаметны в пространственном (RGB) представлении.

Предлагаемый фреймворк состоит из двух частотно–ориентированных ветвей, одна из которых направлена на изучение тонких закономерностей подделки с помощью частотно–ориентированной декомпозиции изображений (FAD), а другая – на извлечение высокоуровневой семантики из локальной частотной статистики (LFS) для описания частотно–ориентированного статистического расхождения между реальными и поддельными лицами.

Блок FAD преобразует изображение в частотную область через DCT (Discrete Cosine Transform), адаптивно разделяет частотный спектр на полосы (низкие, средние, высокие) с помощью обучаемых фильтров, а обратное преобразование (IDCT) генерирует компоненты изображения, выделяющие артефакты в разных частотных диапазонах. Высокочастотные компоненты особенно важны, так как подделки часто оставляют следы в текстурах и границах.

Блок LFS применяет скользящее окно DCT (SWDCT) для анализа локальных участков изображения, собирает статистику (средние значения) по частотным полосам в каждом окне и формирует многоканальную карту, отражающую распределение частот в пространственной области. Данный блок позволяет обнаруживать аномалии в локальных паттернах, устойчивые к сжатию.

В дальнейшем эти две ветви постепенно объединяются с помощью модуля перекрестного внимания, а именно MixBlock, который способствует активному взаимодействию между вышеупомянутыми ветвями FAD и LFS.

Вся модель обнаружения подделок лиц изучается на основе кросс-энтропийной функции потерь. Обширные эксперименты показывают, что технология F3-Net значительно улучшает характеристики по сравнению с некачественными поддельными носителями благодаря тщательному исследованию абляции.

1.3.3. Модели на основе ошибки реконструкции.

Эти методы основаны на предположении, что сгенерированные изображения, прошедшие через процесс диффузии, находятся ближе к латентному пространству диффузионной модели (см. Рисунок 2). Следовательно, они в меньшей степени подвержены отклонению при реконструкции, если их снова подвергнуть процессу шумоподавления диффузионной модели по сравнению с реальными изображениями. Используя ошибку реконструкции, эти методы могут обеспечить удовлетворительную производительность при обнаружении изображений, сгенерированных различными моделями. Однако эти методы по-прежнему сталкиваются с трудностями при обобщении на невидимые наборы данных [34].

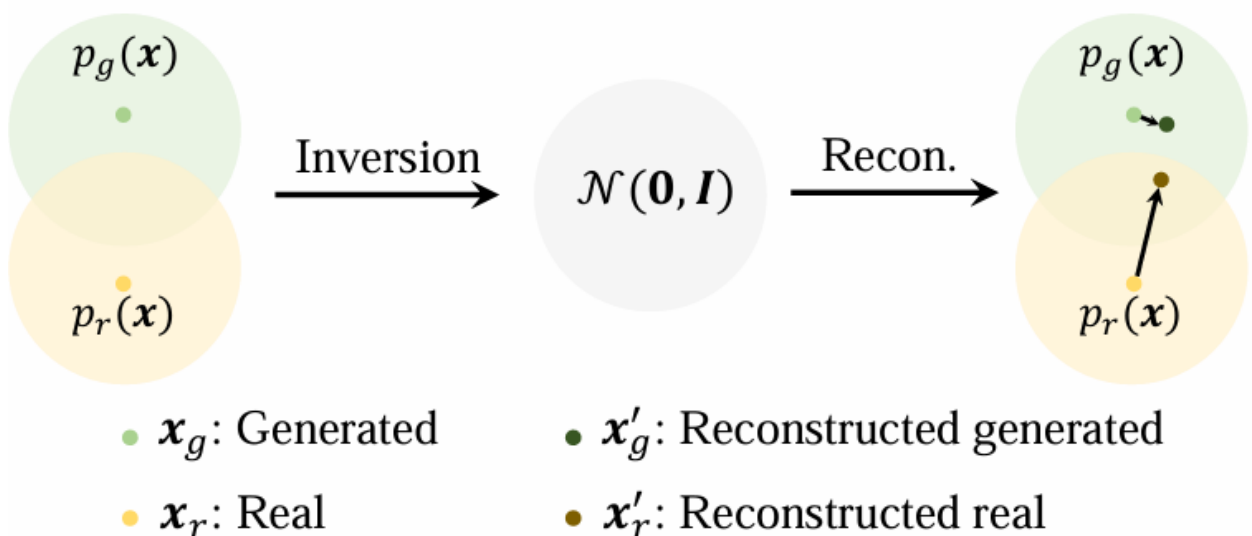


Рисунок 2 – Иллюстрация разницы между распределением реального изображения и реконструированного

Модель DIffusion Reconstruction Error (DIRE). Данная архитектура [35] использует предварительно обученную диффузионную модель для получения реконструкции исходного изображения и исследования разницы между оригинальным изображением и реконструированным для детектирования синтетических изображений (см. Рисунок 3).

Однако, этот метод с трудом справляется со все более развивающимися генеративными моделями, демонстрируя значительные недостатки в возможностях обнаружения из разных наборов данных и разных генераторов.

Также проблемой DIRE является, что требуется многошаговая обратный процесс диффузии (DDIM [12]), который является очень низкоэффективным в реальных задачах.

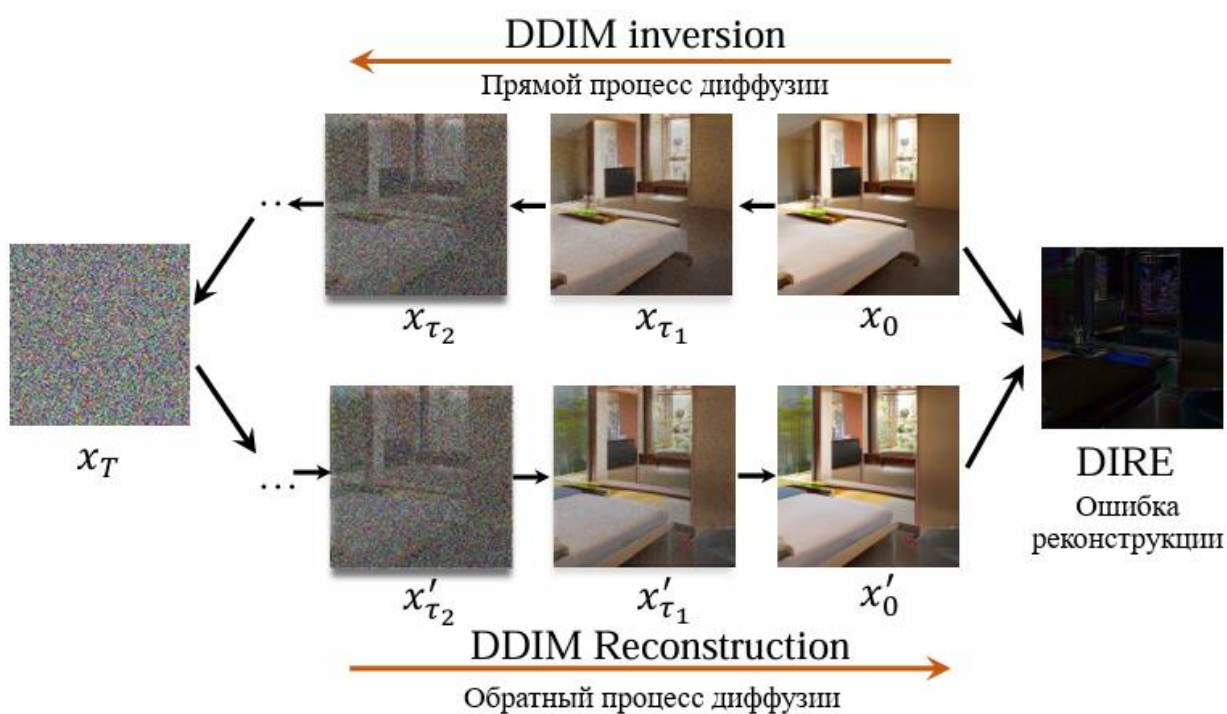


Рисунок 3 – Иллюстрация архитектуры DIRE

Модель DIffusion Noise Feature (DNF). Данная архитектура [36] хоть и не высчитывает ошибку реконструкции, тем не менее использует обратный процесс диффузии для получения новых характеристик об изображении.

Согласно данной архитектуре, начальное изображение проходит обратный процесс диффузии через предварительно обученную диффузионную модель и получаются сэмплы для каждого временного шага диффузионной модели (см. Рисунок 4). Для получения характеристики DNF необходимо выбрать стратегию слияния данных сэмплов и передать в классификатор для распознавания. Стратегия слияния является гиперпараметром в данной архитектуре, как и предварительно обученная диффузионная модель. Но согласно [36] оказывается, что выбор первого сэмпла является лучшей стратегией, с точки зрения качества и производительности модели.

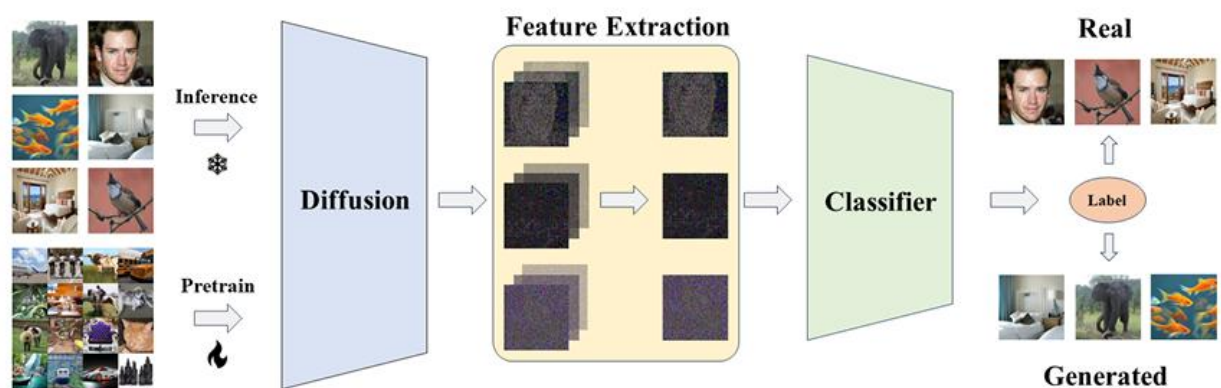


Рисунок 4 – Иллюстрация архитектуры DNF

Модель Frequency-guided Reconstruction Error (FIRE). Данная архитектура [1] является продолжением моделей с использованием ошибки реконструкции.

Данная архитектура основана на гипотезе, что среднечастотные компоненты изображения сложно реконструировать с помощью диффузионной модели. Таким образом, если из реального изображения удалить эти среднечастотные компоненты, ошибка реконструкции будет меньше, чем у неизмененного изображения. Другими словами, за счет удаления некоторой внутренней информации, которая отличает реальные

изображения от сгенерированных, модифицированное реальное изображение становится более похожим на сгенерированное (псевдосгенерированное изображение). Следовательно, разница между реальным изображением и его псевдосгенерированной версией будет больше, чем разница между сгенерированным изображением и его псевдосгенерированной версией. Таким образом, это различие, обусловленное частотой компонента изображения, может служить сигналом для распознавания реального и сгенерированного изображения (см. Рисунок 5 и Рисунок 6).

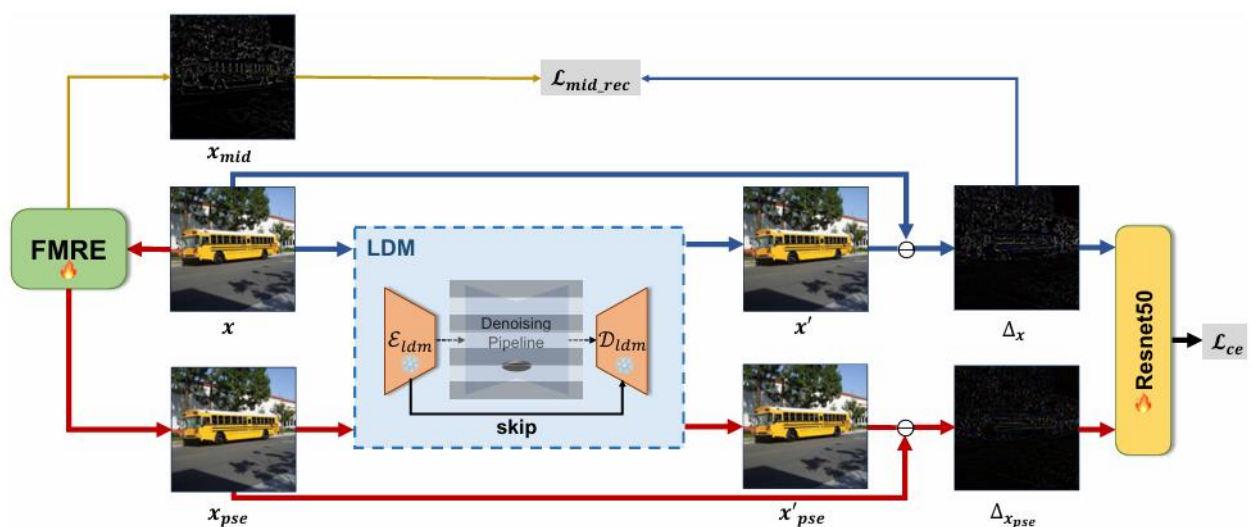


Рисунок 5 – Иллюстрация архитектуры FIRE

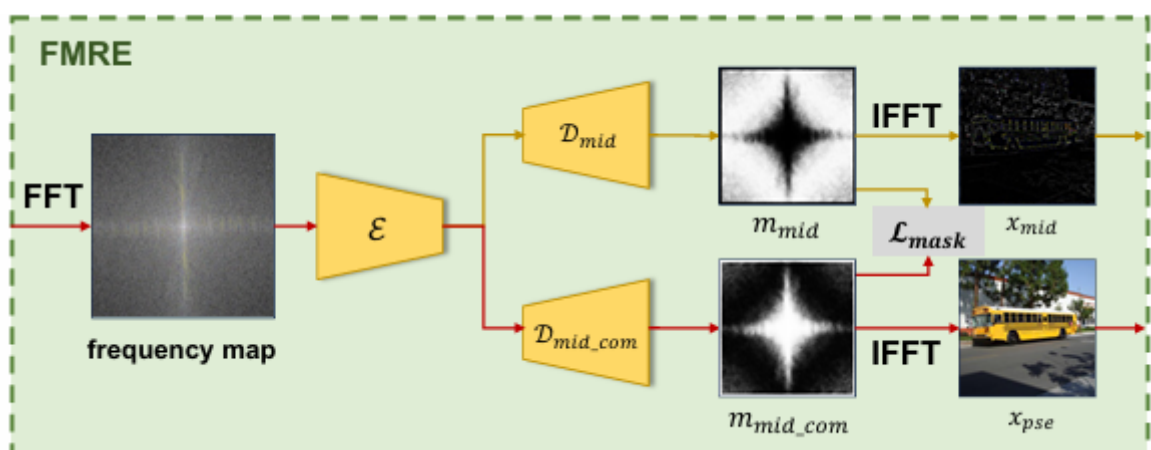


Рисунок 6 – Иллюстрация модуля FMRE из архитектуры FIRE

1.3.4. Модели на основе текстовой информации об изображении.

Генеративные модели преобразования текста в изображение используют обратный процесс диффузии, начиная от случайного шума к выходному изображению и уменьшая шум, основываясь на текстовом промпте [37]. Практически все диффузионные модели (Stable Diffusion, DALL-E, Midjourney и т.д.), используемые сегодня для генерации изображений основываются на этом принципе.

Поэтому вместо того, чтобы просто обучать классификатор на реальных и сгенерированных изображениях, предлагается дополнительно для классификации использовать промпт изображения.

DE-FAKE. Данная архитектура [38] для обучения классификатора использует комбинацию изображения и его промпта (см. Рисунок 7). DE-FAKE представляет собой комплексную систему, направленную на решение трех ключевых задач: обнаружение поддельных изображений, их атрибуция к исходным моделям генерации и анализ влияния текстовых подсказок (prompts) на аутентичность изображений.

Архитектура данной модели – это комбинация эмбеддингов изображений и текстового промпта через CLIP (модель для совместного представления текста и изображения). Изображение сначала кодируется с помощью CLIP Image Encoder, а текстовый промпт кодируется с помощью CLIP Text Encoder. Затем эмбеддинги конкатенируются и подаются на двухслойную полносвязную нейронную сеть для классификации. Модель может использоваться с естественными подсказками (из исходного датасета) или с подсказками, сгенерированными моделью BLIP [39] (если подсказки отсутствуют).

Утверждается, что внедрение промпта в классификацию изображения лучше идентифицирует сгенерированные изображения, т.к. они всегда ближе в пространстве к промпту, чем реальные изображения.

Результаты эксперимента также показали, что данный детектор демонстрирует более высокую точность (например, 0,945 для Stable Diffusion) благодаря усилению различий между реальными и поддельными изображениями через анализ связи "изображение–текст". Также обнаружен общий артефакт в частотных спектрах поддельных изображений (центральная область ярче и более концентрирована).

Дополнительно в статье [38] по данной модели были проанализированы промпты и их влияние на классификацию. Выяснено, что промпты с темой "персона" генерируют более аутентичные изображения, оптимальная длина промпта 25–75 слов, а количество существительных не оказывает значимого влияния на классификацию. Промпты с детальным описанием объекта (например: "собака, выглядывающая из окна машины") повышают аутентичность. А подсказки, описывающие окружение ("зеленый автобус на улице"), упрощают обнаружение подделок.

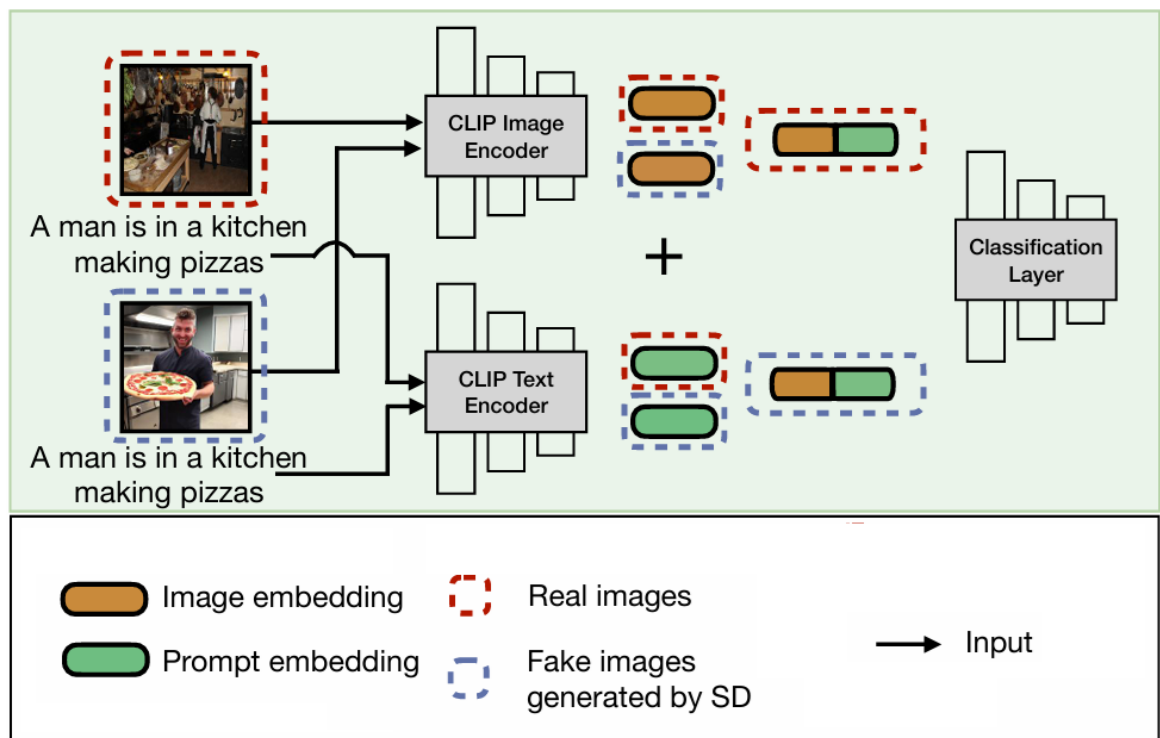


Рисунок 7 – Иллюстрация архитектуры DE-FAKE

ZeroFake. Модель ZeroFake [40] представляет собой метод обнаружения поддельных изображений, созданных или отредактированных текстово–графическими моделями (например, Stable Diffusion [14], DALL–E 2), без необходимости предварительного обучения (zero–shot подход). Её работа основана на анализе устойчивости изображений к возмущениям в процессе DDIM–инверсии (Denoising Diffusion Implicit Models) под воздействием специально сконструированных подсказок противника (adversary prompts).

Основные этапы работы ZeroFake:

1) Генерация обратной подсказки (Reverse Prompt). Для входного изображения x с помощью модели BLIP [39] генерируется текстовая подсказка P_{oracle} , описывающая его содержание. Например, для изображения мотоцикла подсказкой может быть: «A motorcycle parked in a parking space».

2) Создание подсказок противника (Adversary Prompts). Первое существительное в обратной подсказке заменяется на слова из заранее подготовленного списка (например, «tree», «dog», «airplane»). Это создаёт набор искажённых подсказок.

3) Из полученных подсказок выбирается та, которая имеет наибольшее косинусное расстояние от исходной P_{oracle} (т.е. максимально семантически отличается). Например, для подсказки «A motorcycle parked...» заменой первого существительного на «tree» получится «A tree parked...».

4) DDIM–инверсия с подсказкой противника. Изображение кодируется в латентное пространство z_0 с помощью VAE–энкодера и затем к латентному представлению z_0 применяется процесс DDIM–инверсии под управлением выбранной подсказки противника. На этом этапе в изображение последовательно добавляется шум, имитируя обратный процесс диффузии.

5) Восстановление изображения. Полученный зашумленный латентный код реконструируется обратно в изображение x' через процесс денойзинга, также управляемый подсказкой противника.

б) Сравнение исходного и восстановленного изображений. Вычисляется метрика структурного сходства SSIM между x и x' . Если SSIM превышает заданный порог τ , изображение классифицируется как поддельное, иначе – как настоящее.

Поддельные изображения, созданные через диффузионные модели, демонстрируют большую устойчивость к возмущениям во время DDIM-инверсии. Это связано с их синтетической природой: они изначально генерируются через аналогичные процессы добавления и удаления шума. Восстановленные поддельные изображения сохраняют больше исходных деталей, тогда как настоящие сильнее искажаются из-за конфликта между их естественной структурой и направляющей подсказкой противника.

Преимуществами ZeroFake является то, что не требуется обучения на размеченных данных, работает с изображениями от любых текстово-графических моделей, сохраняет точность даже при добавлении шума, размытия или других искажений и эффективно обнаруживает поддельные художественные работы и отредактированные изображения.

Таким образом, ZeroFake использует подходы ошибки реконструкции и текстовых промптов, что помогает лучше улавливать фундаментальные различия в реальных и синтетических изображениях в диффузионных процессах.

1.4. Выводы по главе 1

В данной главе был проведен анализ существующих архитектур и методов, применяемых для детектирования сгенерированных изображений. Данный анализ показывает, что, несмотря на значительные успехи в области генерации изображений, задача их детектирования остается сложной и требует дальнейших исследований. Разработка новых методов, которые смогут учитывать как визуальные, так и частотные и текстовые характеристики изображений, а также адаптироваться к постоянно совершенствующимся

технологиям генерации, является ключевым направлением для обеспечения безопасности и защиты от злоупотреблений синтетическим контентом.

Глава 2. Исследование и анализ выбранной архитектуры и методологии

2.1. Описание проводимого эксперимента

В данной работе проводится исследование эффективности изменения существующего алгоритма детектирования изображений, сгенерированных диффузионными моделями.

Идея эксперимента состоит в том, чтобы оценить изменение метрик эффективности модели классификации изображений на валидационной выборке.

Предлагается взять существующую модель FIRE [1] и внедрить возможность модели использовать сгенерированный описательный промпт изображения в качестве дополнительной характеристики при классификации изображения.

Предлагается следующая схема экспериментов:

- обучить модель FIRE на стандартном наборе данных;
- добавить дополнительную характеристику в модель;
- обучить обновленную модель FIRE' на том же наборе данных;
- сравнить метрики качества классификации.

2.2. Описание и подготовка набора данных

2.2.1. Описание набора данных

В данной задаче используется набор данных GenImage [41], который является одним из стандартов для обучения моделей детектирования сгенерированных изображений и оценки эффективности данных моделей. Данный набор содержит изображения большего разрешения и количества нежели CIFAKE [42], основанный на CIFAR наборе данных.

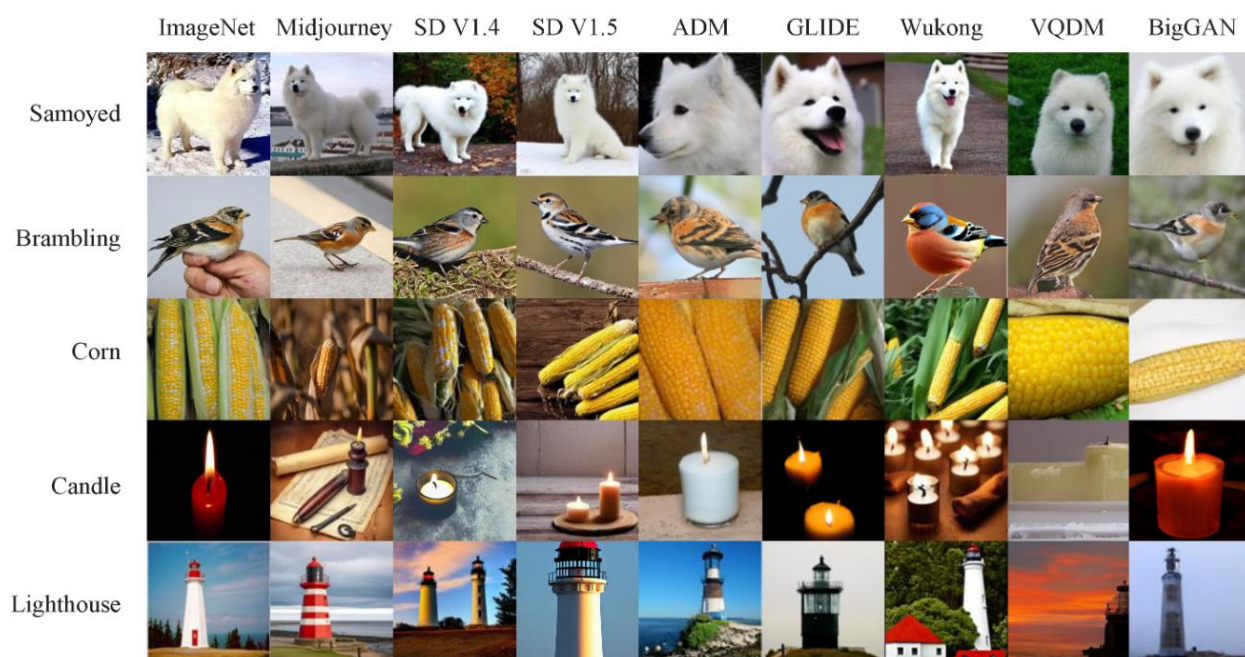


Рисунок 8 – Пример набора данных GenImage

Набор данных содержит более миллиона пар «сгенерированное изображение/реальное изображение». Основан на реальных изображениях из ImageNet [43] и поэтому изображения представлены в тех же 1000 классов, как в ImageNet.

Использованы самые продвинутые генераторы изображений: Midjourney, Stable Diffusion, ADM, GLIDE, Wukong, VQDM, BigGAN.

Таблица 1 – Используемые генераторы изображений в GenImage

Модель генератора	Краткое описание модели
Stable Diffusion	диффузионная модель для генерации изображений высокого качества на основе текстовых описаний с упором на настройку и использование на локальных устройствах
Midjourney	популярный генератор изображений, специализирующийся на стилизованных и художественных иллюстрациях, создаваемых из

Модель генератора	Краткое описание модели
	текстовых запросов и использующий технологии диффузионных моделей
ADM	модель, основанная на процессе адаптивного удаления шума и предназначенная для генерации изображений с использованием диффузионных методов, которые адаптируются к данным на этапе обучения, что позволяет достигать впечатляющих результатов.
GLIDE	модель, объединяющая диффузионные методы и язык для генерации и редактирования изображений из текстовых описаний.
VQDM	комбинирует векторное квантование и диффузионные процессы для повышения качества и стабильности генерации.
Wukong	модель, разработанная для текстово–графических задач, с акцентом на китайский язык и генерацию изображений с азиатской стилистикой.
BigGAN	Генеративно–состязательная сеть, адаптированная для создания изображений с высоким разрешением и детализацией.

Ввиду ограниченности ресурсов для обучения были оставлены только по 5000 изображений для каждой модели (4000 на тренировочную и 1000 на тестовую выборки). Также с целью проведения эксперимента в обучении были оставлены только диффузионные модели генерации изображений [44] и дополнительно удалена модель Wukong. Модель Wukong опирается на семантику китайского языка, что может исказить наши методы. Также дополнительно на данной модели, будет проверена возможность наших

методов к обобщению и нахождению признаков для детектирования сгенерированных изображений.

2.2.2. Подготовка набора данных

Эксперимент построен на принципе бинарной классификации, т.е. сравнение класса, предсказанного моделью и действительного класса. Реальные изображения помечаются классом «1», а сгенерированные классом «0».

Данные разделены на тренировочную и валидационную выборки в соотношении 80% и 20% соответственно (см. Таблица 2).

Таблица 2 – Распределение и использование данных

	Тренировочный набор		Валидационный набор	
	Реальные	Сгенерированные	Реальные	Сгенерированные
VQDM	2000	2000	500	500
SD v1.5	2000	2000	500	500
ADM	2000	2000	500	500
GLIDE	2000	2000	500	500
Midjourney	2000	2000	500	500
Всего:	10000	10000	2500	2500

Дополнительные параметры аугментации, которые могут изменить распределение или частотность компонентов изображения (изменение размеров, поворот, изменение яркости, размытие, наложение шумов, и.т.д.), в данной работе не применяются, т.к. необходимо сохранить как можно больше характеристики исходного изображения для обучения классификатора.

В качестве аугментации используется только случайное переворачивание по горизонтали и центральное кадрирование для того, чтобы изображение соответствовало размеру модели.

Для устойчивой работы диффузионных процессов важно, чтобы распределение изображений было нормальным. Поэтому в предобработке изображений используется нормализация со средним и стандартным отклонением, используемым в ImageNet: $([0,485, 0,456, 0,406], [0,229, 0,224, 0,225])$.

2.3. Описание выбранной модели для проведения исследования

Для проведения эксперимента были проанализированы модели детектирования сгенерированных изображений и выбрана модель FIRE (см. раздел 1.3), как самая современная и наилучшими показателями детектирования на различных наборах данных.

2.3.1. Описание основной модели FIRE

Схема архитектуры модели представлены на Рисунок 5 и Рисунок 6. Код представлен в ПРИЛОЖЕНИИ А. Алгоритм данной модели выглядит следующим образом:

Основная структура:

- 1) Выбирается изображение x из партии данных;
- 2) Проводим данное изображение через модуль Frequency Mask Refinement Module (FMRE) и получаем изображения x_{pse} (отфильтрованы среднечастотные компоненты изображения) и x_{mid} (остались только среднечастотные компоненты). Подробное описание модуля FMRE см. ниже в соответствующем подразделе.

3) Начальное изображение x и отфильтрованное x_{pse} проходят через VAE от диффузионной модели («runwayml/stable-diffusion-v1-5») и получаются реконструкции изображений x' и x_{pse}' .

4) Вычисляются ошибки реконструкции:

$$\Delta x = |x' - x|, \quad (3)$$

$$\Delta x_{pse} = |x_{pse}' - x_{pse}|, \quad (4)$$

5) Полученные ошибки реконструкции направляются в предобученную сверточную нейронную сеть resnet50 [45] для классификации.

Модуль FMRE:

1) Принимается изображение x ;
2) Преобразуем полученное изображение и получаем частотный спектр выполнением на нём FFT.

3) Частотный спектр преобразуется кодировщиком для получения латентное представления изображения.

4) Латентное представление проходит параллельно через два декодировщика (D_{mid} и D_{mid_c}), которые выдают следующие фильтрационные маски:

– m_{mid} – фильтр, чтобы оставить только среднечастотные компоненты изображения;

– m_{mid_c} – фильтр, чтобы отфильтровать среднечастотные компоненты и оставить все остальные частоты;

5) Накладываем полученные маски на частотный спектр и затем обратным преобразованием IFFT получаются два отфильтрованных изображения:

– x_{mid} – остались только среднечастотные изображения (фильтр m_{mid});

– x_{pse} – остались все, кроме среднечастотных компонентов (фильтр m_{mid_c}).

2.3.2. Описание обновленной модели FIRE+BLIP для проведения эксперимента.

Для улучшения качества модели предлагается добавить в схему классификации параметр текстового описания изображения. Ввиду того, что большинство диффузионных моделей (например Stable Diffusion [2; 24] , Midjourney [15], используют текстового описания для генерации изображения, поэтому предполагается, что описание больше подходит для генерированных моделей, нежели реальных, что поможет при классификации. Код для данной модели представлен в ПРИЛОЖЕНИИ Б.

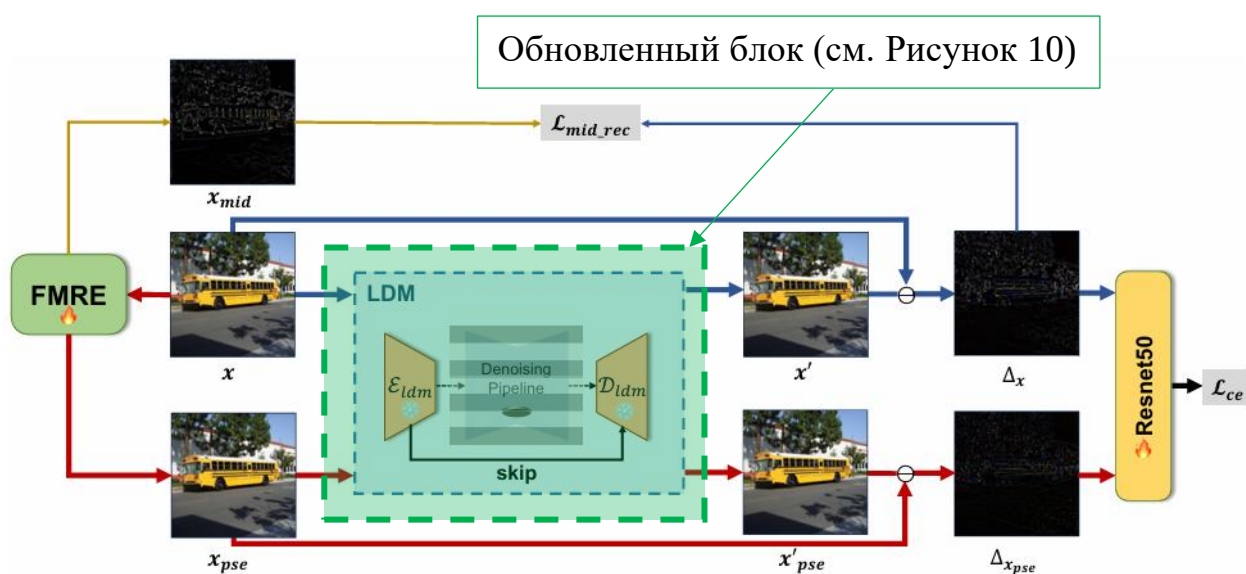


Рисунок 9 – Схема обновления модели FIRE

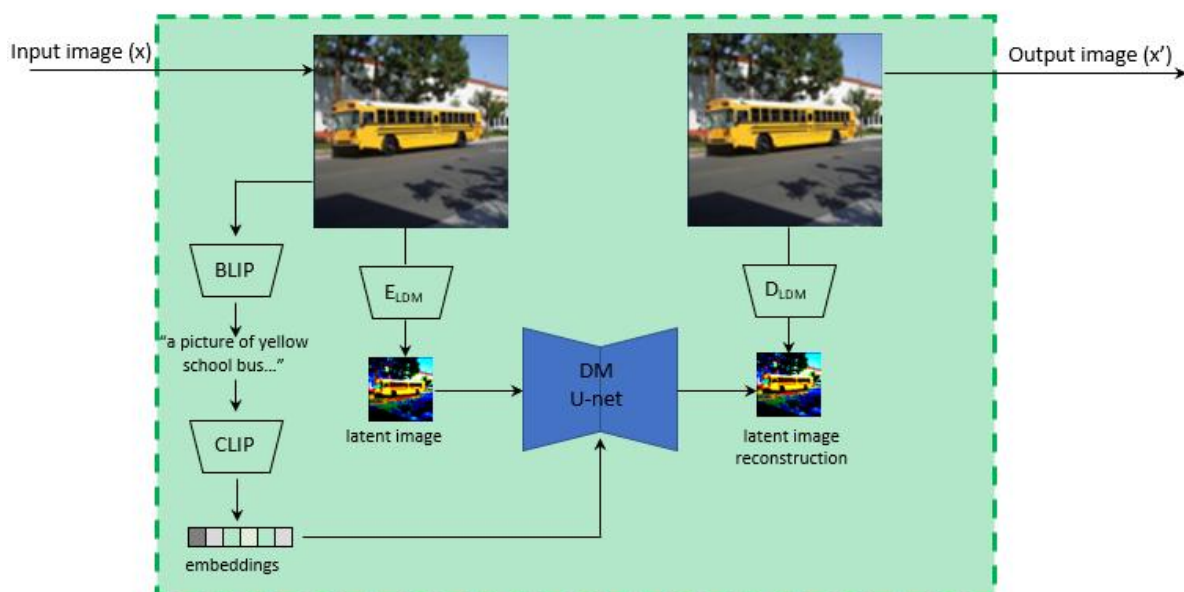


Рисунок 10 – Обновленный блок модели FIRE

Основным изменением является добавление эмбедингов текстового описания изображения. Диффузионная модель, используемая в данной схеме теперь преобразует изображения по следующему механизму:

Модуль BLIP:

- 1) Получаем исходное изображение x .
- 2) Преобразуем изображение в его латентное представление, проведя через энкодер VAE диффузионной модели.
- 3) Получаем текстовое описание изображения проведя его через BLIP. В качестве BLIP используется модель «Salesforce/blip-image-captioning-base» [46] из репозитория HuggingFace.
- 4) Получаем эмбединги текстового описания с помощью модели CLIP. В качестве модели CLIP используется текстовый кодировщик, который используется диффузионной моделью «runwayml/stable-diffusion-v1-5» для генерации изображений.
- 5) Латентное представление изображения проходит через диффузионный процесс с помощью U-net модели из DM с добавлением полученных эмбедингов.

б) Реконструированное латентное представление после модель U-net, проходит через декодер и получается реконструированное изображение.

2.4. Обучение модели и методы оценки

2.4.1. Используемые функции потерь и гиперпараметры модели

Для процесса обучения модели используются следующие функции потерь:

а) L_{CE} – бинарная кросс-энтропия, для обучения классификатора на «реальные» и «сгенерированные» изображения.

б)

$$L_{mask} = \|m_{mid} - M_{mid}\|_2^2 + \|m_{mid_c} - M_{mid_c}\|_2^2 + \|1 - m_{mid} - m_{mid_c}\|_2^2, \quad (5)$$

данная функция потерь отвечает за приближение обученных масок фильтрации (m_{mid} и m_{mid_c}) к начальным (M_{mid} и M_{mid_c}), в которых фильтруются средние частоты / низко-высоких частоты, а также, чтобы они были дополнением друг друга (сумма масок была 1);

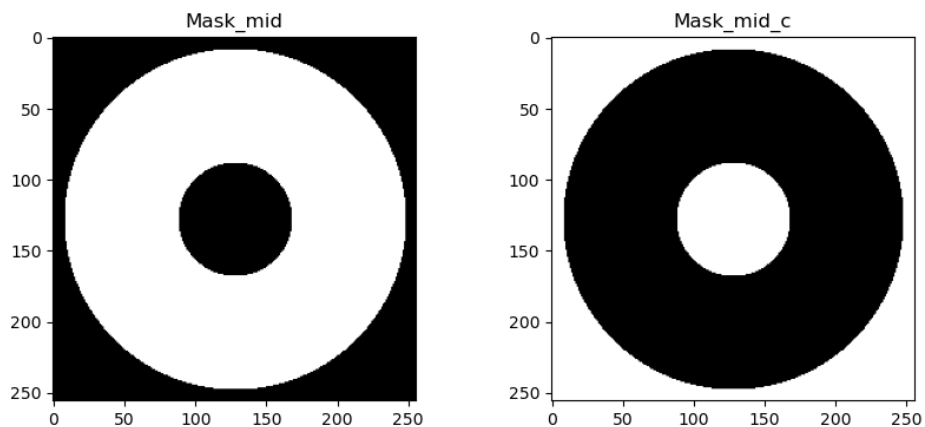


Рисунок 11 – Начальные маски M_{mid} и M_{mid_c}

в)

$$L_{mid_{rec}} = \|x_{mid} - \Delta_x\|_2^2, \quad (6)$$

данная функция потерь приближает ошибку реконструкции Δ_x к изображению, в котором отфильтрованы средние частоты x_{mid} , т.к. диффузионная модель не восстанавливает компоненты средне–частотного диапазона.

Далее данные три функции потерь складываются со своими коэффициентами. В эксперименте используются 3/5 для бинарной кросс–энтропии L_{CE} и по 1/5 для L_{mask} и $L_{mid_{rec}}$.

Также для обучения модели выбраны следующие гиперпараметры (см. Таблица 3):

Таблица 3 – Значения гиперпараметров

Гиперпараметр	Значение
Количество эпох	10
Размер партии	4
Начальная скорость обучения	0,0001
Оптимизатор	Adam [47]
Размер изображения	256 x 256

2.4.2. Метрики оценки эффективности моделей

Для дальнейшей оценки результатов мы определили классификацию на основе четырех возможных состояний: истинно положительное (TP), истинно отрицательное (TN), ложноположительное (FP) и ложноотрицательное (FN). После этого определим метрики, на основании которых будем оценивать эффективность моделей, используя точность (Accuracy), точность (Precision),

полнота (Recall), F1 score, которые являются стандартными метриками в моделях классификации и рассчитываются следующим образом:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}, \quad (7)$$

$$Recall = \frac{TP}{TP + FN}, \quad (8)$$

$$Precision = \frac{TP}{TP + FP}, \quad (9)$$

$$F_1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}, \quad (10)$$

Ассурасу показывает общее количество верных ответов модели. Precision показывает количество истинно положительных исходов из всего набора положительных меток класса. Recall – способность нашей модели избегать ложноотрицательных результатов путем расчета отношения истинно положительных к числу всех изображений, которые действительно принадлежат к данному классу. F1-score показывает средневзвешенное значение Precision и Recall, чтобы обеспечить сбалансированную оценку эффективности модели.

2.4.3. Интервальные оценки метрик классификации

Метрики, описанные в главе 2.4.2, представляют лишь точечные оценки модели, которые практически никогда не будут совпадать с истинным значением точности при непрерывном распределении и множестве вариантов датасетов. Поэтому предлагается использовать интервальные оценки метрик точности модели с 95% доверительным интервалом, чтобы оценить вероятный

интервал разброса метрик моделей для более корректного сравнения улучшения модели.

Для расчета интервальных оценок предлагается использовать метод Вальда, который использовался в статье [48]. Данный метод обычно применяется для построения доверительных интервалов параметров и основывается на асимптотической нормальности оценок. Метод Вальда предполагает, что оценка параметра распределена асимптотически нормально. Тогда доверительный интервал будет иметь вид: «оценка» $\pm z \cdot SE$, где z – это квантиль нормального распределения (например, 1.96 для 95% доверительного интервала), а SE – стандартная ошибка оценки. Следовательно, нужно найти эти стандартные ошибки для каждой метрики.

Ассигасу. Поскольку ассигасу – это доля успехов (правильных классификаций) в биномиальном эксперименте, то стандартная ошибка для ассигасу может быть вычислена как:

$$SE_{acc} = \sqrt{\frac{Accuracy \cdot (1 - Accuracy)}{TP + TN + FP + FN}}, \quad (11)$$

где $TP+TN+FP+FN$ – это общее количество примеров. Тогда доверительный интервал для ассигасу будет:

$$CI_{acc} = Accuracy \pm 1,96 \cdot SE_{acc}, \quad (12)$$

Precision. В precision TP и FP – это количество истинно положительных и ложно положительных результатов. Можно рассматривать precision как условную вероятность $P(\text{реальный положительный класс} \mid \text{предсказанный положительный класс})$. Здесь $TP + FP$ – это все предсказанные положительные классы. Precision – это доля истинно положительных результатов среди всех

предсказанных положительных классов. Тогда стандартную ошибку для precision можно оценить аналогично биномиальной пропорции:

$$SE_{prec} = \sqrt{\frac{Precision \cdot (1 - Precision)}{TP + FP}}, \quad (13)$$

а доверительный интервал:

$$CI_{prec} = Precision \pm 1,96 \cdot SE_{prec}, \quad (14)$$

Recall. Для recall практически тоже самое, что и для precision. В recall используется TP+TN – т.е. все истинные положительные примеры и следовательно стандартная ошибка:

$$SE_{recall} = \sqrt{\frac{Recall \cdot (1 - Recall)}{TP + FN}}, \quad (15)$$

а доверительный интервал:

$$CI_{recall} = Recall \pm 1,96 \cdot SE_{recall}, \quad (16)$$

F1–мера – это гармоническое среднее между precision (P) и recall (R). Для нахождения стандартной ошибки для F1 можно использовать дельта–метод [49] для приближения стандартной ошибки функции случайных величин. F1 зависит от precision и recall, которые сами зависят от TP, FP, FN, поэтому потребуется найти асимптотическую дисперсию F1 через дисперсии precision и recall и их ковариацию:

$$SE_{F1} = \sqrt{\left(\frac{\partial F1}{\partial P}\right)^2 \cdot SE_{prec}^2 + \left(\frac{\partial F1}{\partial R}\right)^2 \cdot SE_{recall}^2 + 2 \cdot \frac{\partial F1}{\partial P} \cdot \frac{\partial F1}{\partial R} \cdot Cov(P, R)}, \quad (17)$$

где

$$\frac{\partial F1}{\partial P} = \frac{2 \cdot Recall^2}{(Precision + Recall)^2}, \quad (18)$$

$$\frac{\partial F1}{\partial R} = \frac{2 \cdot Precision^2}{(Precision + Recall)^2}, \quad (19)$$

Для упрощения будем считать, что Precision и Recall независимые случайные величины (хотя, строго говоря, это не совсем корректно, т.к. они оба зависят от TP), тогда $Cov(P, R) = 0$ и получается:

$$SE_{F1} = \sqrt{\left(\frac{2 \cdot R^2}{(P + R)^2} \cdot SE_{prec}\right)^2 + \left(\frac{2 \cdot P^2}{(P + R)^2} \cdot SE_{recall}\right)^2}, \quad (20)$$

а доверительный интервал:

$$CI_{F1} = F1 \pm 1,96 \cdot SE_{F1}, \quad (21)$$

2.4.4. Оценка качества генерации описания изображения (CLIPScore)

Метрика CLIPScore [50] представляет собой современный подход к оценке качества текстового описания, сгенерированного моделью, в контексте визуального контекста, то есть изображения или видео. Основная цель CLIPScore заключается в использовании принципа согласованности между

текстом и изображением для оценки соответствия описания или подписи содержимому визуального объекта. Эта метрика основывается на модели CLIP, которая была обучена на большом количестве данных, содержащих пары изображений и текстов, с использованием контрастного обучения.

CLIPScore использует возможности модели CLIP для изучения латентного пространства, в котором текстовые и визуальные представления семантически сопоставлены. С помощью этой метрики вычисляется косинусное сходство между текстовым и визуальным представлениями, полученными из CLIP. Чем выше это сходство, тем больше система «считает», что текст соответствует изображению. Таким образом, CLIPScore измеряет способность сгенерированного текста корректно и полно описывать данные, представленные визуальным стимулом:

$$CLIPScore(Image, Caption) = \max(100 \cdot \cos(E_{image}, E_{caption}), 0), \quad (22)$$

Метрика CLIPScore вычисляется следующим образом:

- а) входное изображение и текст (в нашем случае сгенерированное описание) подаются в модель CLIP;
- б) CLIP преобразует изображение в векторное представление (визуальный эмбединг) и текст – в текстовый эмбединг;
- в) вычисляется косинусное сходство между вектором изображения и вектором текста;
- г) полученное значение интерпретируется как CLIPScore.

Как правило, значения нормализуются или ограничиваются в положительном диапазоне (например, от 0 до 100), чтобы было удобнее интерпретировать результаты.

CLIPScore предлагает альтернативу традиционным методам оценки качества текстов, таким как BLEU, ROUGE, CIDEr и другие, которые в основном ориентируются на совпадение ключевых слов или фраз между

эталонными текстами и гипотезами. Эти традиционные методы часто не учитывают контекст или визуально–значимую информацию, что ограничивает их применимость в задачах, связанных с мультимодальностью (например, подписи к изображениям, видео–аннотации).

Преимущество CLIPScore в том, что метрика оценивает смысловую связь между описанием и изображением, игнорируя лексическую точность, что делает ее более «гибкой» в задачах мультимодальной генерации. Также CLIPScore не требует наличия эталонных подписей для оценки качества модели. Это особенно важно в сценариях, где создание больших датасетов вручную сложно или неизбежно сопровождается субъективностью.

В то же время, метрика CLIPScore может быть ограничена набором данных, на которых она была обучена модель CLIP, что может привести к потенциальной смещенности. Например, CLIPScore может переоценивать описания, которые соответствуют доминирующим шаблонам из обучающих данных. Также метрика не всегда учитывает важные для человека аспекты, такие как грамматическая правильность или стилистическая элегантность текста. И если изображение содержит сложные или неоднозначные визуальные элементы, CLIP может неверно интерпретировать их, что повлияет на значение метрики.

С помощью данной метрики CLIPScore будет оцениваться в главе 3 влияние качества сгенерированного описания на корректность классификации.

2.5. Выводы по главе 2

В данной главе разработан и описан подход к проведению эксперимента для проверки гипотезы по улучшению классификации сгенерированных изображений. Был сделан обоснованный выбор архитектуры базовой модели, проработаны детали её модификации для расширения функционала и повышения качества. Вместе с тем детально описаны состав данных,

методология подготовки и критерии оценки, что обеспечит корректное сравнение моделей FIRE и FIRE+BLIP. Внедрение текстовых эмбедингов, согласно ожиданиям, должно привести к значительным улучшениям в эффективности классификации, а применение доверительных интервалов усилит статистическую значимость полученных результатов.

Глава 3. Анализ работы и результаты проведения эксперимента

В данной главе представлены результаты экспериментального исследования, направленного на оценку эффективности модифицированной модели FIRE+BLIP для детектирования сгенерированных изображений. На основании методологии, описанной в Главе 2, проведено сравнение метрик классификации базовой модели FIRE и её обновленной версии, использующей текстовые эмбединги из генеративных промптов. Целью главы является:

- 1) Демонстрация количественных результатов улучшения качества классификации.
- 2) Статистический анализ значимости различий метрик с использованием доверительных интервалов.
- 3) Визуализация и интерпретация ключевых примеров работы моделей.

3.1. Экспериментальная инфраструктура и условия тестирования

Обучение и тестирование обеих моделей проводилось в облачной среде Google Colab и Kaggle Notebook с графическими ускорителями GPU NVIDIA Tesla T4, 16GB RAM.

Код классов моделей (см. ПРИЛОЖЕНИЕ А и ПРИЛОЖЕНИЕ Б) и шагов конвейера обучения написан на Python с использованием фреймворка PyTorch.

Обучение проходило на 10 эпохах. Ввиду размеров датасета и ограниченности наличия производительных ресурсов, обучение каждой эпохи занимало около 3 часов. Тем не менее даже на этих 10 эпохах виден прогресс, что обновленная модель начинает лучше справляться с детектирование сгенерированных изображений.

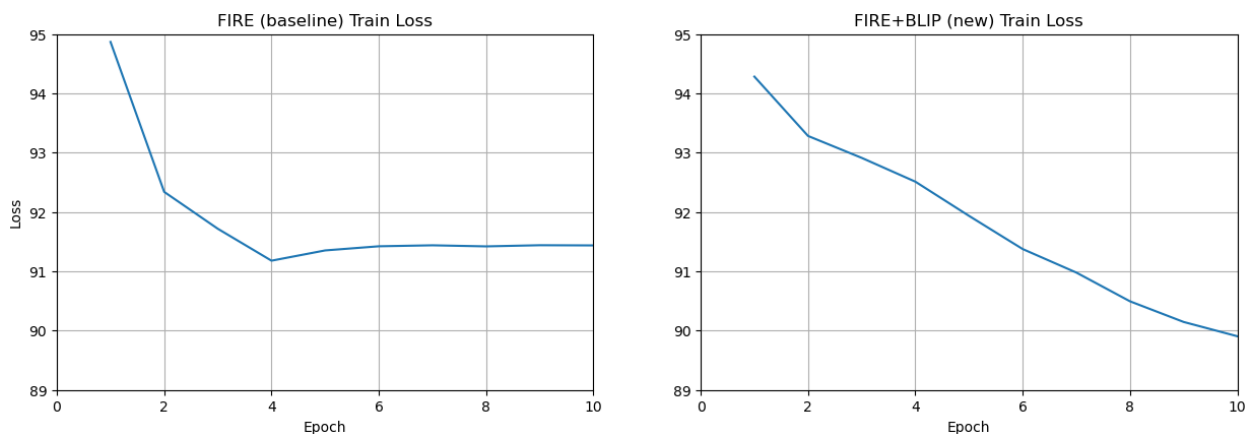


Рисунок 12 – График функции потерь

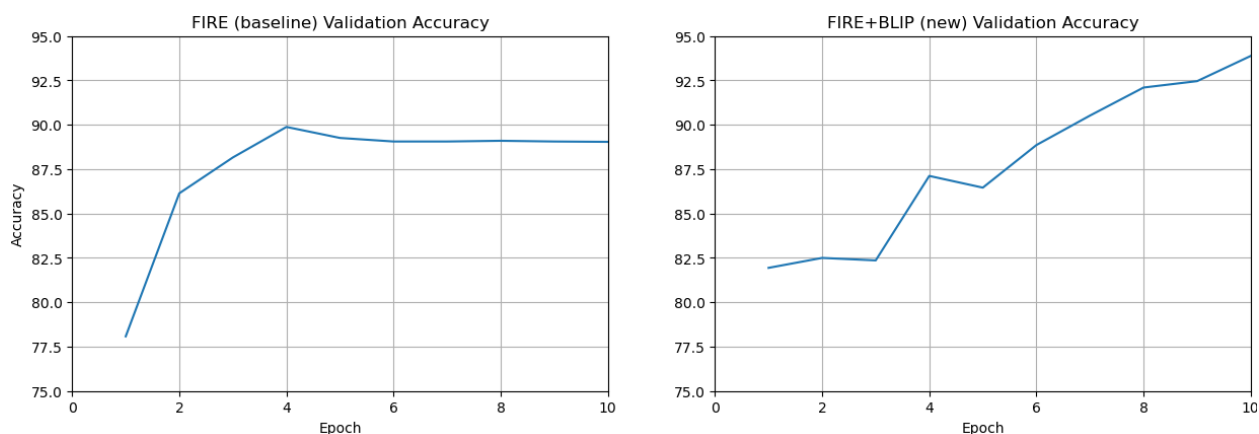


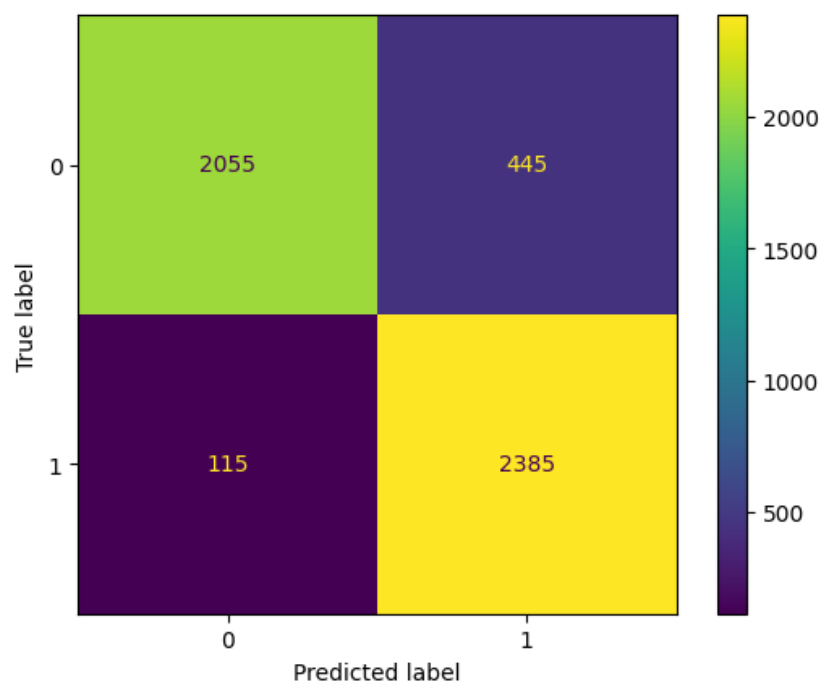
Рисунок 13 – График точности (ассурасу) на тестовой выборке

Из графиков обучения видно, что базовая модель (FIRE) за 5 эпох вышла в зону, в которой больше не уменьшалась функция потерь, что подтверждает достаточность обучения 10 эпох для данной модели. Напротив, у новой модели функция потерь уменьшалась все 10 эпох увеличение числа эпох вероятно бы улучшило метрики новой модели. Но консервативно и из-за ограниченности ресурсов, оставим моделям равные условия для сравнения характеристик.

3.2. Сравнительный анализ метрик классификации.

После обучения обеих моделей получены следующие метрики классификации:

FIRE-model (baseline) confusion matrix



FIRE+BLIP-model (new) confusion matrix

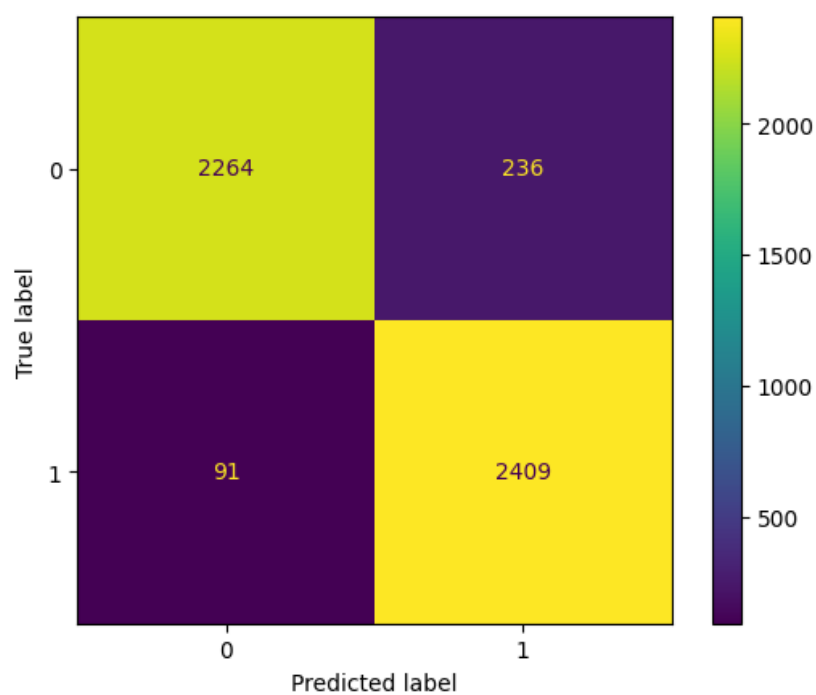


Рисунок 14 – Матрицы ошибок

Таблица 4– Метрики классификации

Метрика	Модель FIRE (базовая)	Модель FIRE+BLIP (новая)
Accuracy	0,8793 – 0,8967	0,9277 – 0,9415
Precision	0,8293 – 0,8562	0,8999 – 0,9216
Recall	0,9458 – 0,9622	0,9563 – 0,9709
F1	0,8866 – 0,9033	0,9297 – 0,9432

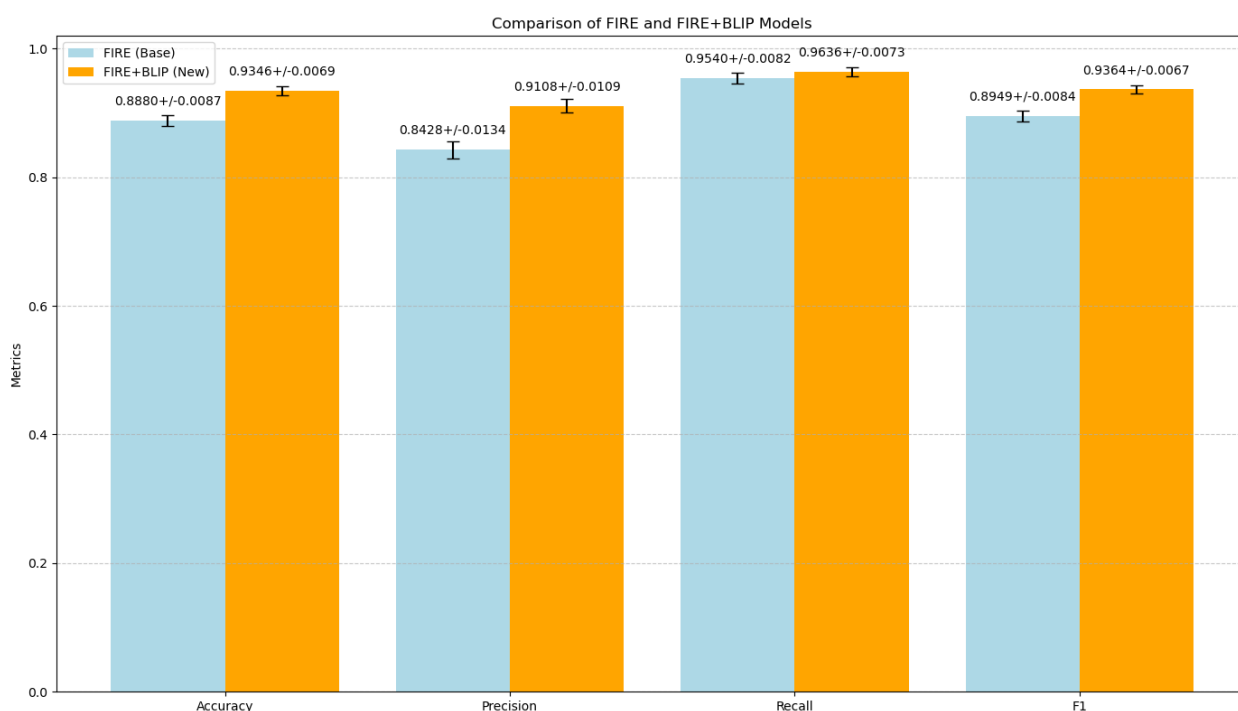


Рисунок 15 – Сравнение метрик классификации

Исходя из полученных значений метрик и доверительных интервалов, можно сказать, что точность (accuracy/precision) и F1–мера новой модели статистически значимо увеличились относительно базовой модели. Полнота (recall) увеличилась незначительно, что говорит о том, что улучшения детектирования реальных изображений не произошло.

Новая модель FIRE+BLIP демонстрирует значимое превосходство над базовой версией по точности (accuracy), precision и F1–мере, о чём свидетельствует отсутствие пересечения доверительных интервалов. Это указывает на устойчивость улучшений и их практическую значимость.

Основной прирост эффективности связан с улучшением распознавания искусственных изображений. Это может объясняться тем, что интеграция BLIP (возможно, как мультимодальной модели) добавила новые признаки или контекст, повышающие чувствительность к артефактам генерации. Снижение числа ложноположительных случаев (FP) для класса «0» напрямую повысило precision, что критично для задач, где ошибочная классификация искусственного контента как реального недопустима (например, модерация).

Незначительный рост recall указывает, что модель сохранила способность корректно идентифицировать реальные изображения, несмотря на оптимизацию для класса «0». Это важно для баланса между минимизацией ошибок первого и второго рода.

Улучшение precision при сохранении recall делает модель более надежной в сценариях, где критично избегать ложных принятий сгенерированного контента (например, борьба с дезинформацией). Однако, если задача требует максимизации обнаружения всех реальных изображений (даже ценой роста FP), может потребоваться дополнительная настройка порогов классификации.

Внедрение архитектуры FIRE+BLIP позволило создать более сбалансированную модель, которая эффективнее решает задачу классификации за счет улучшения детектирования сгенерированного контента. Это подтверждает гипотезу о том, что комбинация методов анализа изображений и мультимодальных подходов повышает качество распознавания, особенно в условиях быстро развивающихся технологий генерации.

3.3. Интерпретация результатов

3.3.1. Анализ влияния текстовых эмбеддингов на ошибки реконструкции.

Выполнено построение гистограмм распределения метрики оценки качества (CLIPScore [50]) генерации описания изображения с помощью BLIP-модели (см. Рисунок 10):

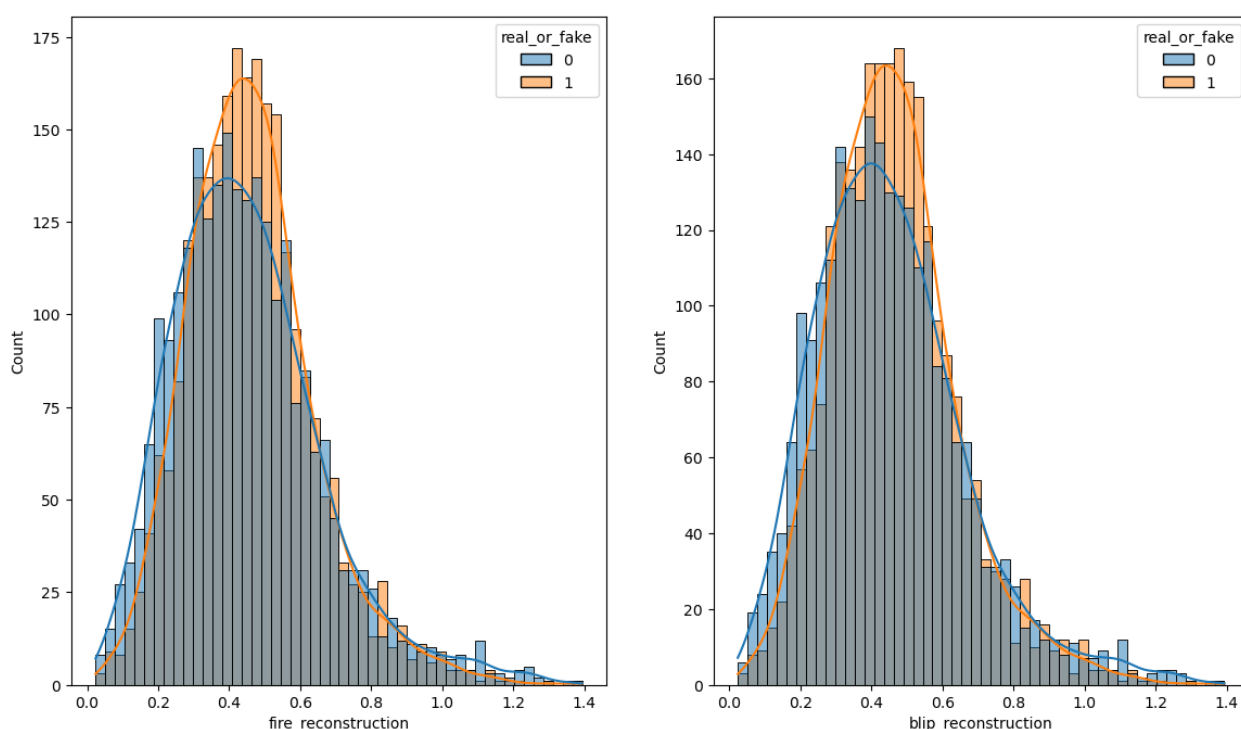


Рисунок 16 – Гистограмма распределения метрики оценки качества генерации описания изображения

Таблица 5 – Характеристики распределения ошибки реконструкции

Модель	Метка класса	Среднее	Ст. отклонение
FIRE	Реальное	0,4619	0,1789
	Сгенерированное	0,4498	0,2106
FIRE+BLIP	Реальное	0,4643	0,1790
	Сгенерированное	0,4520	0,2108

Модель	Метка класса	Среднее	Ст. отклонение
Δ	Реальное	0,0024	—
	Сгенерированное	0,0022	—

Как видно из Рисунок 16 и Таблица 5 можно сделать вывод, что распределения очень похожи и не отличаются. Для подтверждения проведен статистический тест Стьюдента для сравнения средних двух зависимых выборок. Тест для зависимых выборок выбран для оценки изменения данной ошибки реконструкции после внедрения улучшенной модели по отношению к начальному значению у базовой модели:

Листинг 1 – t-тест Стьюдента для зависимых выборок для сгенерированных изображений:

```

statistic, pvalue = ttest_rel(df[df['real_or_fake'] ==
0.0]['fire_reconstruction'], df[df['real_or_fake'] ==
0.0]['blip_reconstruction'])
print(f"pvalue = {pvalue:.4f} -> ", end='')
if pvalue < 0.05:
    print(f'Отвергаем H0 (средние не изменились) как
противоречащую экспериментальным данным, следовательно, средние
изменились')
else:
    print(f'Нет оснований отвергнуть H0 - средние не изменились')
>>> pvalue = 0.0000 -> Отвергаем H0 (средние не изменились) как
противоречащую экспериментальным данным, следовательно, средние
изменились

```

Листинг 2 – t-тест Стьюдента для зависимых выборок для реальных изображений:

```

statistic, pvalue =
ttest_rel(df_standard[df_standard['real_or_fake'] ==
1.0]['fire_reconstruction'],

```

```

df_standard[df_standard['real_or_fake']
1.0]['blip_reconstruction']
print(f"pvalue = {pvalue:.4f} -> ", end='')
if pvalue < 0.05:
    print(f'Отвергаем H0 (средние не изменились) как
противоречащую экспериментальным данным, следовательно, средние
изменились')
else:
    print(f'Нет оснований отвергнуть H0 - средние не изменились')
>>> pvalue = 0.0000 -> Отвергаем H0 (средние не изменились) как
противоречащую экспериментальным данным, следовательно, средние
изменились

```

Обе нулевые гипотезы, что средние значения ошибки реконструкции для реальных и сгенерированных изображений не изменились в новой модели были отвергнуты, как противоречащие экспериментальным данным. Это означает, что в среднем ошибки реконструкции статистически значимо изменились (увеличились) после внедрения новой модели, что означает, что данный призрак стал лучше показывать разделения между реальными и сгенерированными изображениями т.к. увеличение ошибки реконструкции на реальных изображениях произошло больше.

Также выполнено построение гистограммы распределения оценки качества (CLIPScore [50]) генерации описания изображения моделью BLIP:

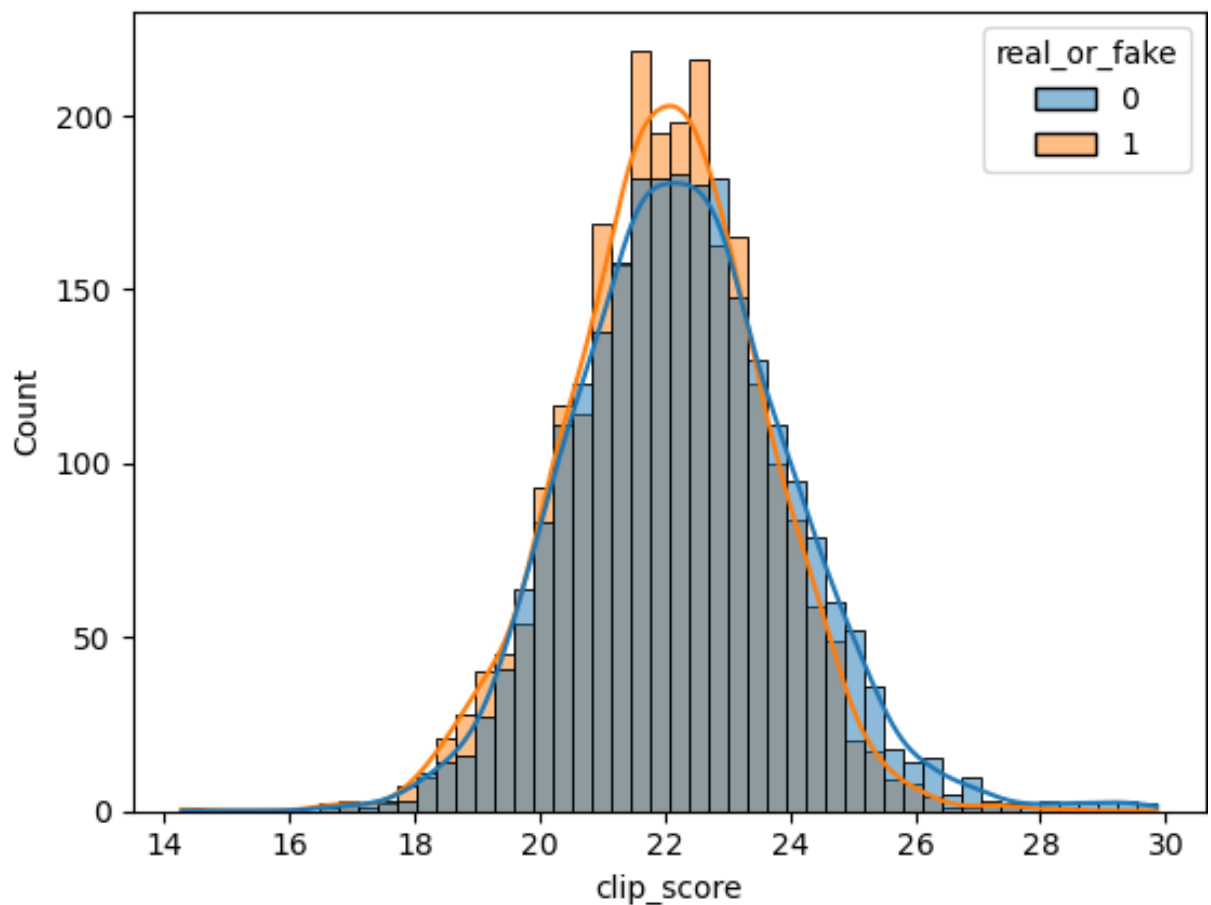


Рисунок 17 – Гистограмма распределения оценки качества описания изображения

Таблица 6 – Характеристики распределения оценки качества описания

Показатель	Метка класса	Среднее	Ст. отклонение
CLIPScore	Реальное	22,00	1,56
	Сгенерированное	22,29	1,75

Также дополнительно проведем t–тест Стьюдента для независимых выборок для сравнения средних значений оценки для реальных и сгенерированных изображений:

Листинг 3 – t-тест Стьюдента для независимых выборок для реальных изображений:

```
statistic, pvalue =
ttest_ind(df_standard[df_standard['real_or_fake']
1.0]['clip_score'], df_standard[df_standard['real_or_fake']
0.0]['clip_score'])
print(f"pvalue = {pvalue:.4f} -> ", end='')
if pvalue < 0.05:
    print(f'Отвергаем H0 (средние не изменились) как
противоречащую экспериментальным данным, следовательно, средние
изменились')
else:
    print(f'Нет оснований отвергнуть H0 - средние не изменились')
>>> pvalue = 0.0000 -> Отвергаем H0 (средние не изменились) как
противоречащую экспериментальным данным, следовательно, средние
изменились
```

Из Листинг 3 видно, что нулевая гипотеза о том, что среднее значение оценки качества генерации описания неизменно для реальных и сгенерированных изображений, не подтвердилась. Следовательно, можно считать, что чем лучше сгенерированное описание подходит к картинке, тем более вероятно, что эта картинка сгенерированная, а не реальная.

3.3.2. Сравнение эффективности моделей для разных генераторов

Обе модели (базовая и новая) обучались на одном и том же датасете (см. Раздел 2.2). Для обучения были оставлены только изображения, сгенерированные диффузионными моделями (удалена модель BigGAN). Также, были удалены изображения, сгенерированные моделью Wukong. Это выполнено намерено, чтобы проанализировать как наши модели смогут обучиться обобщать новые (не встречавшиеся в обучении) данные (изображения), сгенерированные новым генератором.

Для проверки наших моделей, был построен график точностей (ассигасу) на тестовых изображениях всех имеющихся генераторов (включая те, которые модели не видели при обучении):

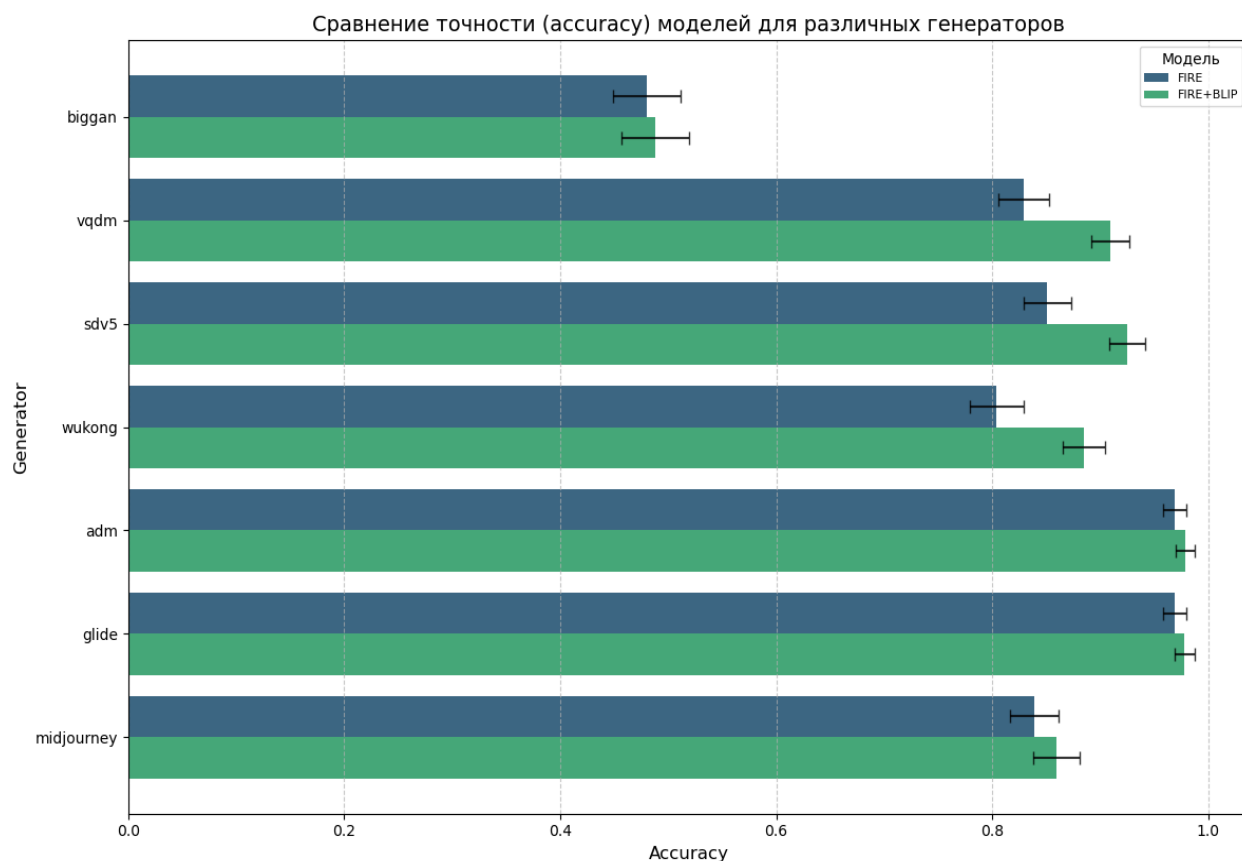


Рисунок 18 – График точности (ассигасу) для разных генераторов

Таблица 7 – Значения точности и доверительного интервала для изображений различных генераторов

Генератор	Используемая модель	Точность (ассигасу)
ADM	FIRE (базовая)	0,969 ± 0,011
	FIRE+BLIP (новая)	0,979 ± 0,009
BigGAN	FIRE (базовая)	0,480 ± 0,031
	FIRE+BLIP (новая)	0,488 ± 0,031
GLIDE	FIRE (базовая)	0,969 ± 0,011
	FIRE+BLIP (новая)	0,978 ± 0,009
Midjourney	FIRE (базовая)	0,839 ± 0,023

Генератор	Используемая модель	Точность (accuracy)
	FIRE+BLIP (новая)	$0,859 \pm 0,022$
Stable Diffusion 1.5	FIRE (базовая)	$0,851 \pm 0,022$
	FIRE+BLIP (новая)	$0,925 \pm 0,016$
VQDM	FIRE (базовая)	$0,829 \pm 0,023$
	FIRE+BLIP (новая)	$0,909 \pm 0,017$
Wukong	FIRE (базовая)	$0,804 \pm 0,024$
	FIRE+BLIP (новая)	$0,885 \pm 0,019$

Из Рисунок 13 и Таблица 7 можно видеть, что точность (accuracy) выросла на наборе изображений всех видов генераторов, включая даже те, которые модель не видела в процессе обучения (модели BigGAN и Wukong).

Значительный прирост точности на Stable Diffusion 1.5 и VQDM указывает на то, что интеграция текстовых метаданных (BLIP) особенно полезна для генераторов, чьи артефакты коррелируют с семантикой изображений. Это позволяет предположить, что текстовые описания помогают выявлять несоответствия между визуальным контентом и скрытыми паттернами генерации, характерными для этих моделей.

Точность на модели Wukong выросла статистически значимо. Улучшение точности на данных Wukong, несмотря на их отсутствие в обучении, свидетельствует о том, что FIRE+BLIP научилась выделять универсальные признаки, общие для диффузионных моделей. Текстовая информация, вероятно, усилила способность модели к обобщению, снизив зависимость от специфики отдельных генераторов.

Что касается модели BigGAN, то её точность слишком низкая и даже ниже 50% (случайное «угадывание»). Крайне низкая точность на BigGAN подтверждает, что артефакты GAN принципиально отличаются от диффузионных. Это требует разработки гибридных подходов, сочетающих методы детекции для GAN (например, анализ спектральных особенностей

CNNDetector [32], см. обзор в Разделе 1.3) и диффузионных моделей (анализ семантико–текстовых несоответствий).

На некоторых моделях (Midjourney, ADM, GLIDE) точность значимо не выросла. Что может говорить, что базовая модель смогла выявить нужные признаки и добавление текстового описания не улучшило эти признаки для классификации.

Исключение BigGAN и Wukong из обучающей выборки могло создать смещение в сторону диффузионных моделей. Также эксперименты проводились на ограниченном наборе генераторов и требуется валидация на более разнообразных данных (например, DALL–E).

Новая модель FIRE+BLIP демонстрирует прогресс в детекции синтетических изображений, особенно для диффузионных генераторов. Однако её эффективность ограничена для GAN–архитектур, что подчеркивает необходимость адаптивных мультимодальных решений в условиях быстро развивающихся технологий генеративного искусственного интеллекта.

3.3.3. Корреляционный анализ зависимостей признаков

Корреляция улучшения значения предсказания и качества сгенерированного описания (BLIP). Для начала построим диаграмму рассеяния (scatter plot) между значением качества оценки сгенерированного описания (CLIPScore [50]) и значения улучшения оценки «delta_output». Данное значение является разницей выходных данных (до активации сигмойдой) между новой (FIRE+BLIP) и базовой (FIRE) моделями. И учтён тот факт, что будет положительным при приближении к «правильному» классу и отрицательным при приближении к «неправильному» классу.

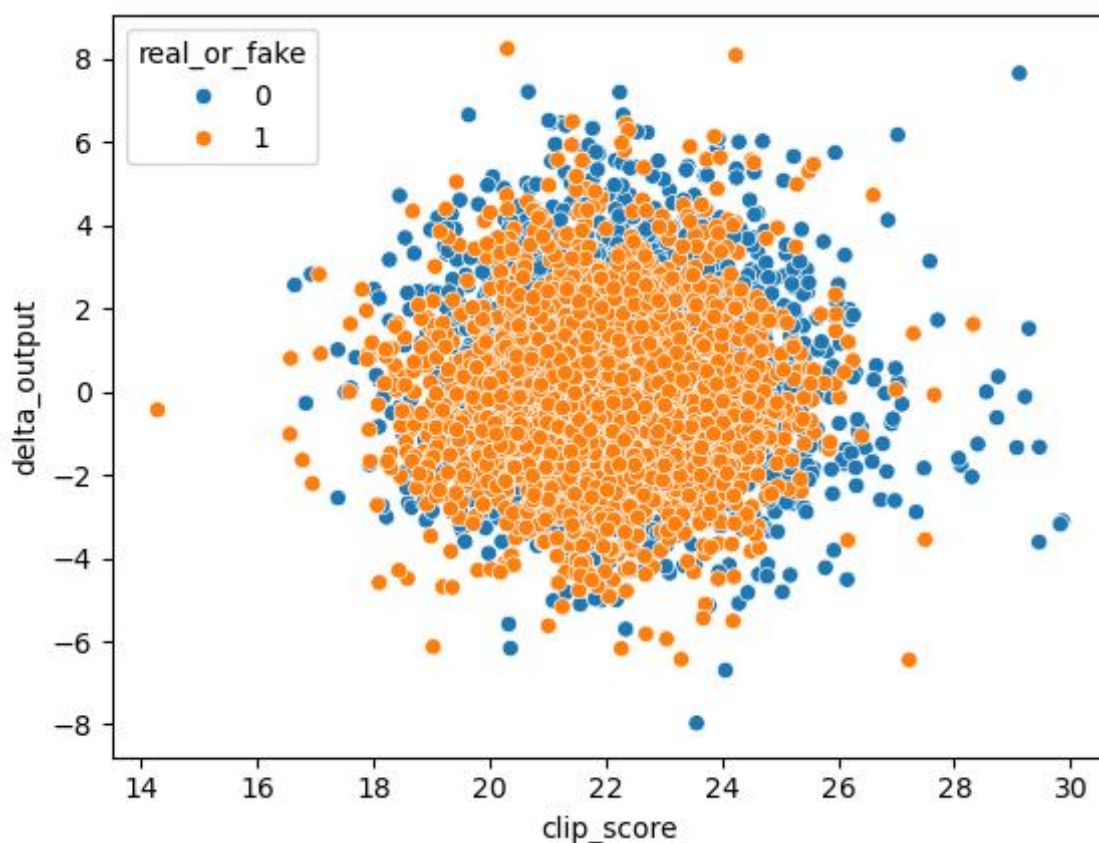


Рисунок 19 – Диаграмма рассеяния улучшения значения предсказания от качества сгенерированного описания (CLIPScore)

Рисунок 19 показывает, что значения распределены хаотично и нет зависимости между ними. Проведём корреляционный анализ Пирсона для проверки нулевой гипотезы, что корреляции нет.

Листинг 4 – Корреляционный анализ CLIPScore/Delta_output:

```
r_statistic,      pvalue      =      pearsonr(df['clip_score'],
df['delta_output'])
print(f"pvalue = {pvalue:.4f} -> ", end='')
if pvalue < 0.05:
    print(f'Отвергаем H0 (отсутствие корреляции), следовательно
корреляция существует, r = {r_statistic}')
else:
    print(f'Нет оснований отвергнуть H0 - корреляции нет')
>>> pvalue = 0.4229 -> Нет оснований отвергнуть H0 - корреляции
нет
```

Данный анализ также показывает, что действительно линейной зависимости между данным параметрами нет и улучшение точности модели произошло из-за других факторов.

Корреляция выходного сигнала модели и ошибки реконструкции. Также для визуализации построим диаграмму рассеяния данных признаков:

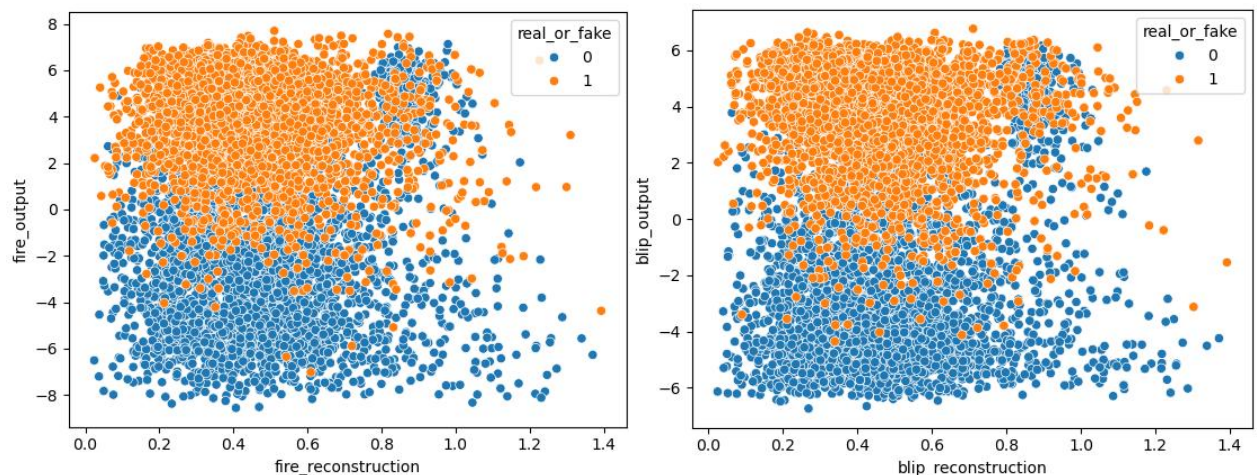


Рисунок 20 – Диаграмма рассеяния выходного сигнала моделей от ошибок реконструкции

На вид по Рисунок 20 также кажется, что зависимости между данным параметрами нет. Для подтверждения проведем корреляционный анализ:

Листинг 5 – Корреляционный анализ Reconstruction Error/Output

```
r_statistic, pvalue = pearsonr(df['fire_reconstruction'],
df['fire_output'])
print(f"pvalue = {pvalue:.4f} -> ", end='')
if pvalue < 0.05:
    print(f'Отвергаем H0 (отсутствие корреляции), следовательно
корреляция существует, r = {r_statistic}')
else:
    print(f'Нет оснований отвергнуть H0 - корреляции нет')
>>> pvalue = 0.0091 -> Отвергаем H0 (отсутствие корреляции),
следовательно корреляция существует, r = 0.06251289496826343
```

Листинг 6 – Корреляционный анализ Reconstruction Error/Output

```
r_statistic, pvalue = pearsonr(df['blip_reconstruction'],
df['blip_output'])
print(f"pvalue = {pvalue:.4f} -> ", end='')
if pvalue < 0.05:
    print(f'Отвергаем H0 (отсутствие корреляции), следовательно
корреляция существует, r = {r_statistic}')
else:
    print(f'Нет оснований отвергнуть H0 - корреляции нет')
>>> pvalue = 0.0000 -> Отвергаем H0 (отсутствие корреляции),
следовательно корреляция существует, r = 0.0934820427083271
```

Корреляционный анализ (Листинг 5 и Листинг 6) показал, что всё-таки наблюдается слабая линейная зависимость, между ошибкой реконструкции и выходным сигналом. Причем подтверждается, что данная связь прямо пропорциональная, т.е. при увеличении ошибки реконструкции увеличивается и выходной сигнал, что означает приближении к классу «реальных» изображений. Что подтверждает предположение о том, что диффузионной модели легче восстановить сгенерированное изображение (меньшая ошибка реконструкции), чем реальное (большая ошибка реконструкции). Причем, у новой модели (FIRE+BLIP) это зависимость чуть сильнее, что подтверждает более лучшую архитектуру данной модели.

Корреляция выходного сигнала модели и оценки качества сгенерированного описания. Для визуализации построим диаграмму рассеяния данных признаков:

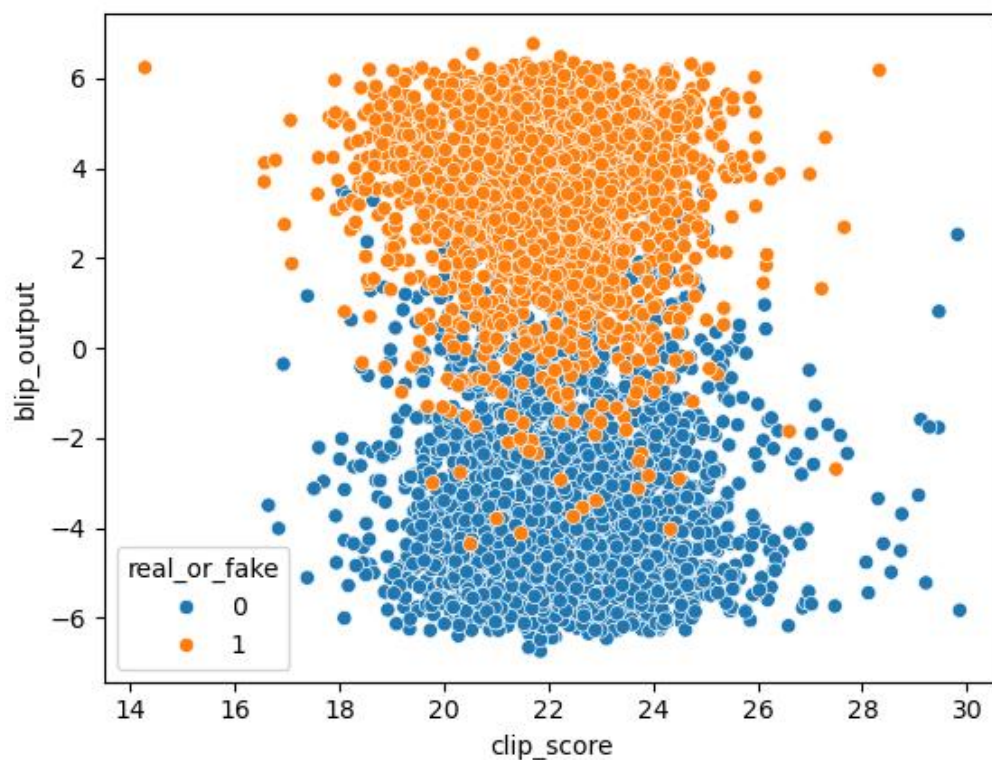


Рисунок 21 – Диаграмма рассеяния выходного сигнала модели от качества сгенерированного описания (CLIPScore)

Явной зависимости между данными параметрами также не наблюдается из данного графика. Корреляционный анализ был проведен только для обновленной модели (FIRE+BLIP), т.к. в базовой не использовались текстовые описания изображений.

Листинг 7 – Корреляционный анализ CLIPScore/Output

```
r_statistic, pvalue = pearsonr(df['clip_score'],
df['blip_output'])
print(f"pvalue = {pvalue:.4f} -> ", end='')
if pvalue < 0.05:
    print(f'Отвергаем H0 (отсутствие корреляции), следовательно
корреляция существует, r = {r_statistic}')
else:
    print(f'Нет оснований отвергнуть H0 - корреляции нет')
>>> pvalue = 0.0000 -> Отвергаем H0 (отсутствие корреляции),
следовательно корреляция существует, r = -0.08673058325793223
```


Данный корреляционный анализ (Листинг 7) показывает, что существует слабая отрицательная линейная зависимость между выходным сигналом модели и качеством, сгенерированного текстового описания. Причем, что характерно, данная зависимость отрицательная, т.е. при положительном увеличении значения оценки качества текстового описания значение выходного сигнала будет уменьшаться, а следовательно, приближаться к классу «сгенерированный» изображений. Т.е. получается, что более качественные описания получаются для сгенерированных изображений.

3.4. Визуализация результатов

3.4.1. Примеры изображений с минимальной и максимальной ошибкой реконструкции

Рассмотрим и проанализируем изображения с минимальной и максимальной ошибкой реконструкции:



Рисунок 22 – 10 изображений с минимальной ошибкой реконструкции

В Таблица 8 представлены конкретные значения 10 минимальных ошибок реконструкции обеих моделей:

Таблица 8 – Значения минимальной ошибки реконструкции

ID	Generator	isReal	Reconstruction (FIRE)	Reconstruction (FIRE+BLIP)
2888	glide	0	0,023	0,024
4534	sdv5	1	0,025	0,025
2983	glide	0	0,037	0,039
2381	adm	0	0,036	0,040
5696	adm	1	0,039	0,041
6843	midjourney	1	0,042	0,043
769	vqdm	0	0,049	0,048
2857	glide	0	0,048	0,049
2580	glide	0	0,051	0,049
2417	adm	0	0,049	0,051

Минимальные значения реконструкции для обеих моделей (FIRE и FIRE+BLIP) достигаются на одних и тех же изображениях и в большинстве случаев эти изображения сгенерированные. Данные изображения обладают размытыми контурами и фоном, что показывает, что данные изображения легче восстанавливать диффузионной модели.



Рисунок 23 – 10 изображений с максимальной ошибкой реконструкции

В Таблица 9 представлены конкретные значения 10 максимальных ошибок реконструкции обеих моделей:

Таблица 9 – Значения максимальной ошибки реконструкции

ID	Generator	isReal	Reconstruction (FIRE)	Reconstruction (FIRE+BLIP)
5937	adm	1	1,393	1,392
2544	glide	0	1,370	1,371
2758	glide	0	1,340	1,338
6282	glide	1	1,299	1,302
2526	glide	0	1,288	1,287
2850	glide	0	1,272	1,273
694	vqdm	0	1,257	1,256
722	vqdm	0	1,254	1,255
2702	glide	0	1,247	1,250
2868	glide	0	1,241	1,240

Максимальные значения реконструкции для обеих моделей (FIRE и FIRE+BLIP) также достигаются на одних и тех же изображениях и также в основном эти изображения сгенерированные. Характерной особенностью

больших ошибок реконструкции являются изображения с отсутствующим фоном (наличие только белого фона), т.к. диффузионная модель пытается восстановить полное изображение и следовательно разница в фонах дает эти большие расхождения. Вероятно для дальнейшей оценки и настройки обучения моделей требуется отфильтровать изображения с нереалистичным или абстрактным фоном.

3.4.2. Примеры изображений с исправленной классификацией

Рассмотрим и проанализируем изображения, для которых была «исправлена» (как в правильную, так и в неправильную сторону) классификация после внедрения новой модели. Добавлен новый параметр модели – «delta_output», который показывает разницу между выходными сигналами (до активации сигмой) между новой (FIRE+BLIP) и базовой (FIRE) моделями, причем если изменение в сторону правильного класса, то данный параметр положительный, иначе отрицательный.



Рисунок 24 – 10 изображений с «правильным» изменением класса

Таблица 10 – Значения 10 корректно исправленных классификаций, отсортированных в порядке уменьшения параметра «delta_output»

ID	Generator	isReal	Output (FIRE)	Output (FIRE+BLIP)	Delta Output
772	vqdm	0	6,543	−2,225	8,769
1346	sdv5	0	4,492	−4,184	8,677
3393	midjourney	0	4,587	−3,945	8,532
1317	sdv5	0	5,501	−2,741	8,243
1498	sdv5	0	5,483	−2,744	8,227
4866	sdv5	1	−1,568	6,179	7,748
5908	adm	1	−7,020	0,145	7,166
973	vqdm	0	3,295	−3,772	7,067
4389	vqdm	1	−2,981	4,014	6,996
6120	glide	1	−4,213	2,756	6,969

Приведенные на Рисунок 24 изображения были базовой моделью классифицированы неправильно, а новая модель исправила классификацию. Каких-либо закономерностей не видно, единственно, что можно заметить, это то, что большинство исправлений для сгенерированных изображений.



Рисунок 25 – 10 изображений с «неправильным» изменением класса

Таблица 11 – Значения 10 некорректно исправленных классификаций, отсортированных в порядке увеличения параметра «delta_output»

ID	Generator	isReal	Output (FIRE)	Output (FIRE+BLIP)	Delta Output
2658	glide	0	−7,566	0,122	−7,688
4281	vqdm	1	5,259	−2,351	−7,611
3498	midjourney	0	−6,651	0,607	−7,258
6518	midjourney	1	5,359	−1,334	−6,693
3026	midjourney	0	−6,594	0,065	−6,659
6368	glide	1	5,712	−0,784	−6,496
4508	sdv5	1	4,232	−2,232	−6,465
6588	midjourney	1	5,068	−0,981	−6,050
6230	glide	1	4,730	−1,280	−6,010
4990	sdv5	1	4,635	−1,114	−5,750

На Рисунок 25 представлены изображения, которые наоборот базовой моделью были классифицированы правильно, а новая модель исправила классификацию на неправильную. Также заметно, что большинство из них – это реальные изображения. Это косвенно подтверждает, что внедрение текстового описания (модуля BLIP) приближает картинки к классу «сгенерированных» изображений (см. раздел 3.3.3).

3.4.3. Примеры работы масок изображений

Рассмотрим частотные фильтры m_{mid} (см. раздел 2.3 и 2.4.1), которые модель выучила на тренировочной выборке. Данный фильтр используется, чтобы оставить только среднечастотные компоненты у изображения и сравнить затем отфильтрованное изображение с ошибкой реконструкции, т.к. диффузионному процессу крайне сложно восстанавливать среднечастотные компоненты. Поэтому, предполагается, что разница между начальным

изображением и реконструкцией как раз и должна отличаться на эти среднечастотные компоненты, которые отсеиваются фильтром.

Рассмотрены данные фильтры отдельно для каждого диффузионного генератора и усреднены по всем изображениям:

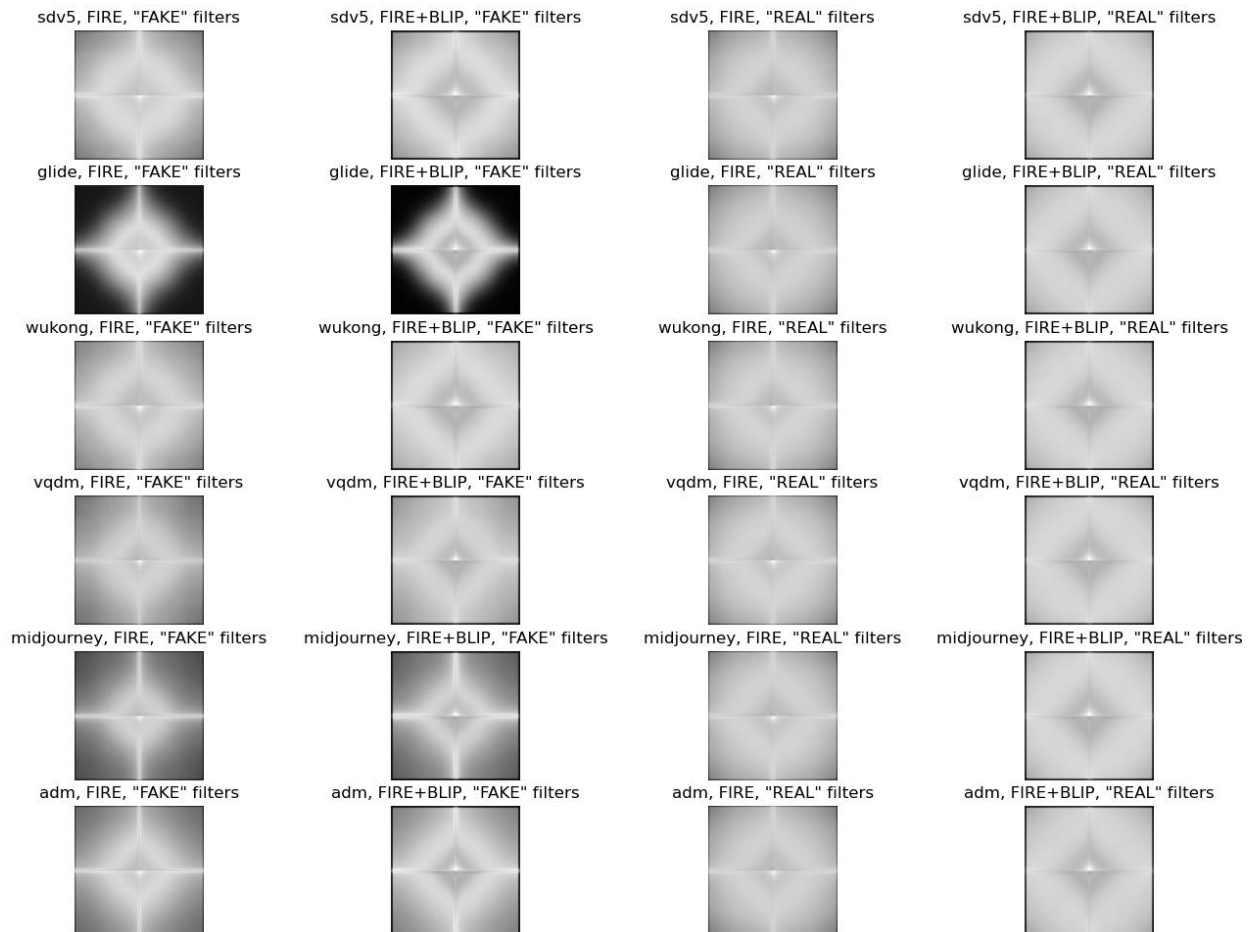


Рисунок 26 – Фильтр m_{mid} моделей FIRE и FIRE+BLIP

Первое, что замечается, что у всех «реальных» изображений практически ничего не фильтруется, причем для обеих моделей. Это означает, что ошибки реконструкции очень разнообразны и не дают какой-то устойчивый шаблон, по которому они отличаются от начальных изображений, поэтому модель и фильтр фактически «оставляют» реальные изображения без изменений.

Следующее, что заметно, что для модели GLIDE сильно выражен фильтр, причем в основном фильтруется (удаляются) высокочастотные

компоненты изображения. Получается, что изображения, сгенерированные данной моделью, очень сильно расходятся в спектре высоких частот с реконструкцией. Также видно, что границы фильтра для новой модели (FIRE+BLIP) более чёткие, что означает, что дополнительная текстовая информация уточнила границы данного фильтра.

Для остальных диффузионных генераторов практически однотипные фильтры, которые более сильней затевают высокочастотные компоненты и чуть слабее затевают низкочастотные компоненты, оставляя более сильный акцент на средних частотах изображений. Это подтверждает схожую природу генерации изображений для всех диффузионных моделей и сложность реконструкции изображений ими именно в средне– и высокочастотном спектрах.

3.5. Выводы по главе 3

Модифицированная модель FIRE+BLIP продемонстрировала статистически значимое улучшение метрик accuracy, precision и F1–меры по сравнению с базовой версией FIRE. Это подтверждает гипотезу о том, что интеграция текстовых эмбеддингов из генеративных промптов (BLIP) усиливает способность модели выявлять артефакты синтетических изображений. При этом сохранение уровня recall свидетельствует о балансировке между минимизацией ошибок I и II рода.

Статистический анализ (t–тесты) показал значимое увеличение ошибок реконструкции для обоих классов, что указывает на усиление разделимости признаков. Корреляция между CLIPScore и выходным сигналом FIRE+BLIP подтвердила, что сгенерированные изображения получают более семантически согласованные описания, что используется моделью для детекции. Однако отсутствие прямой линейной зависимости между CLIPScore и улучшением предсказаний подчеркивает комплексный характер влияния текстовых данных.

Модель FIRE+BLIP показала рост ассигасы для генераторов, не включенных в обучение (Wukong), что свидетельствует о её способности выделять универсальные паттерны диффузионных моделей. А низкая точность на BigGAN подтверждает фундаментальные различия в артефактах GAN и диффузионных подходов, требующие гибридных методов детекции.

Анализ частотных фильтров (m_{mid}) выявил, что FIRE+BLIP эффективнее фокусируется на средне- и высокочастотных компонентах, критичных для детекции артефактов диффузии. Примеры с экстремальными ошибками реконструкции показали, что модели лучше восстанавливают сгенерированные изображения с размытым фоном, но испытывают трудности с абстрактными или монохромными областями.

ЗАКЛЮЧЕНИЕ

Данное исследование основывается на актуальной проблеме выявления изображений, созданных с помощью современных диффузионных моделей, которые по качеству и реалистичности превосходят более ранние генеративные подходы.

Целью данной работы было исследование архитектур и алгоритмов, позволяющие детектировать изображения, сгенерированные с помощью диффузионных моделей, и разработка более эффективного алгоритма детектирования.

Поставленную цель, можно считать выполненной, так как исследование показало статистически значимый прирост в оценках качества модели детектирования сгенерированных изображений.

В первой главе проведён анализ современных методов генерации и детекции синтетического контента, выявлены ключевые проблемы, связанные с высокой реалистичностью изображений от диффузионных моделей, и обоснована необходимость разработки новых подходов. Во второй главе разработана методология эксперимента, включающая модификацию модели FIRE, подготовку набора данных GenImage и выбор метрик оценки, а также предложен алгоритм обучения с учётом частотных и текстовых признаков. В третьей главе продемонстрировано статистически значимое улучшение метрик классификации для модифицированной модели FIRE+BLIP, подтверждена её способность к обобщению на незнакомые генераторы (например, Wukong), а также выявлены ограничения в детекции изображений от GAN-архитектур.

В ходе исследования была подтверждена гипотеза о том, что модификация архитектуры FIRE за счёт интеграции текстовых эмбеддингов, полученных с помощью модели BLIP, позволяет повысить эффективность детектирования изображений, сгенерированных диффузионными моделями.

В ходе выполнения магистерской диссертации были выполнены следующие задачи:

- проведен обзор современных методов детектирования сгенерированных изображений;
- сформирована архитектура и алгоритм для детектирования сгенерированных изображений;
- выбран и подготовлен набор данных для обучения и оценки качества алгоритма;
- проведены эксперименты по обучению детекторов сгенерированных изображений;
- проанализированы результаты экспериментов, метрики и визуализации работы моделей.

В результате выполнения магистерской диссертации осуществлена разработка более эффективного алгоритма детектирования изображений, сгенерированных диффузионными моделями.

Разработанный алгоритм решает задачи идентификации синтетического контента с высокой точностью, превосходя существующие аналоги за счёт комбинации частотного анализа и мультимодальных данных, что подтверждено тестированием на разнородных наборах данных. Перспективы работы включают расширение функционала модели для поддержки гибридных генераторов (GAN и DM), оптимизацию вычислительной эффективности, интеграцию методов устойчивости к атакам на изображения, а также разработку приложений реального времени для обнаружения сгенерированных изображений. Результаты исследования вносят вклад в обеспечение цифровой безопасности и борьбу с дезинформацией, предлагая инструмент для автоматизированного контроля синтетического визуального контента.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. FIRE: Robust Detection of Diffusion–Generated Images via Frequency–Guided Reconstruction Error. FIRE / B. Chu [и др.] // arXiv:2412.07140 [cs]. – 2025. – pp. 1–14.
2. Дипфейки могут сделать отягчающим обстоятельством при совершении преступления [сайт] // Коммерсантъ. – URL: <https://www.kommersant.ru/doc/7516126> (дата обращения: 27.04.2025).
3. Ho J. Denoising Diffusion Probabilistic Models / J. Ho, A. Jain, P. Abbeel // arXiv:2006.11239 [cs]. – 2020. – pp. 1–25.
4. Dhariwal P. Diffusion Models Beat GANs on Image Synthesis / P. Dhariwal, A. Nichol // arXiv:2105.05233 [cs]. – 2021. – pp. 1–44.
5. Korshunov P. DeepFakes: a New Threat to Face Recognition? Assessment and Detection. DeepFakes / P. Korshunov, S. Marcel // arXiv:1812.08685 [cs]. – 2018. – pp. 1–5.
6. On the detection of synthetic images generated by diffusion models / R. Corvi [и др.] // arXiv:2211.00680 [cs]. – 2022. – pp. 1–6.
7. Towards the Detection of Diffusion Model Deepfakes / J. Ricker [и др.] // arXiv:2210.14571 [cs]. – 2024. – pp. 1–12.
8. Generative Adversarial Nets / I. Goodfellow [и др.] // Advances in Neural Information Processing Systems. – Curran Associates, Inc., 2014. – Т. 27. – pp. 1–9.
9. Schneider F. ArchiSound: Audio Generation with Diffusion. ArchiSound / F. Schneider // arXiv:2301.13267 [cs]. – 2023. – pp. 1–48.
10. Generative Time Series Forecasting with Diffusion, Denoise, and Disentanglement / Y. Li [и др.] // arXiv:2301.03028 [cs]. – 2023. – pp. 1–29.
11. Фостер Д. Генеративное глубокое обучение. Как не мы рисуем картины, пишем романы и музыку / Д. Фостер. – 2–е межд изд. – Астана: Спринт Бук, 2024. – 448 с.

12. Song J. Denoising Diffusion Implicit Models / J. Song, C. Meng, S. Ermon // arXiv:2010.02502 [cs]. – 2022. – pp. 1–22.
13. High-Resolution Image Synthesis with Latent Diffusion Models / R. Rombach [и др.] // 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). – 2022. – pp. 10674–10685.
14. Stability AI Image Models [сайт] // Stability AI. – URL: <https://stability.ai/stable-image> (дата обращения: 08.03.2025).
15. Midjourney [сайт] // Midjourney. – URL: <https://www.midjourney.com/home> (дата обращения: 08.03.2025).
16. DIP: Diffusion Learning of Inconsistency Pattern for General DeepFake Detection. DIP / F. Nie [и др.] // arXiv:2410.23663 [cs]. – 2024. – pp. 1–13.
17. Diffusion Deepfake / C. Bhattacharyya [и др.] // arXiv:2404.01579 [cs]. – 2024. – pp. 1–28.
18. Exposing digital image forgeries by illumination color classification / T.J. De Carvalho [и др.] // IEEE Transactions on Information Forensics and Security. – 2013. – Т. 8. – № 7. – pp. 1182–1194.
19. McCloskey S. Detecting GAN-generated Imagery using Color Cues / S. McCloskey, M. Albright // arXiv:1812.08247 [cs]. – 2018. – pp. 1–7.
20. Development of Photo Forensics Algorithm by Detecting Photoshop Manipulation using Error Level Analysis [сайт] // CoLab. – URL: [https://colab.ws/articles/10.11591/ijeecs.v7.i1.pp131–137](https://colab.ws/articles/10.11591/ijeecs.v7.i1.pp131-137) (дата обращения: 10.03.2025).
21. Detection of GAN-Generated Fake Images over Social Networks / F. Marra [и др.] // 2018 IEEE Conference on Multimedia Information Processing and Retrieval (MIPR) 2018 IEEE Conference on Multimedia Information Processing and Retrieval (MIPR). – 2018. – pp. 384–389.

22. Yu N. Attributing Fake Images to GANs: Learning and Analyzing GAN Fingerprints. Attributing Fake Images to GANs / N. Yu, L. Davis, M. Fritz // arXiv:1811.08180 [cs]. – 2019. – pp. 1–41.
23. Detecting GAN generated Fake Images using Co–occurrence Matrices / L. Nataraj [и др.] // arXiv:1903.06836 [cs]. – 2019. – pp. 1–6.
24. McCloskey S. Detecting GAN–Generated Imagery Using Saturation Cues / S. McCloskey, M. Albright // 2019 IEEE International Conference on Image Processing (ICIP) 2019 IEEE International Conference on Image Processing (ICIP). – 2019. – pp. 4584–4588.
25. Zhang X. Detecting and Simulating Artifacts in GAN Fake Images / X. Zhang, S. Karaman, S. –F. Chang // 2019 IEEE International Workshop on Information Forensics and Security (WIFS) 2019 IEEE International Workshop on Information Forensics and Security (WIFS). – 2019. – pp. 1–6.
26. CNN–generated images are surprisingly easy to spot... for now / S. –Y. Wang [и др.] // arXiv:1912.11035 [cs]. – 2020. – pp. 1–13.
27. Leveraging Frequency Analysis for Deep Fake Image Recognition / J. Frank [и др.] // arXiv:2003.08685 [cs]. – 2020. – pp. 1–17.
28. Durall R. Watch your Up–Convolution: CNN Based Generative Deep Neural Networks are Failing to Reproduce Spectral Distributions. Watch your Up–Convolution / R. Durall, M. Keuper, J. Keuper // arXiv:2003.01826 [cs]. – 2020. – pp. 1–14.
29. Rissanen S. Generative Modelling With Inverse Heat Dissipation / S. Rissanen, M. Heinonen, A. Solin // arXiv:2206.13397 [cs]. – 2023. – pp. 1–54.
30. Schwarz K. On the Frequency Bias of Generative Models / K. Schwarz, Y. Liao, A. Geiger // arXiv:2111.02447 [cs]. – 2021. – pp. 1–18.
31. SSD–GAN: Measuring the Realness in the Spatial and Spectral Domains. SSD–GAN / Y. Chen [и др.] // arXiv:2012.05535 [cs]. – 2020. – pp. 1–9.
32. PeterWang512/CNNDetection: Code for the paper: CNN–generated images are surprisingly easy to spot... for now [сайт] // GitHub. – URL: <https://github.com/PeterWang512/CNNDetection> (дата обращения: 10.03.2025).

33. Thinking in Frequency: Face Forgery Detection by Mining Frequency-aware Clues. Thinking in Frequency / Y. Qian [и др.] // arXiv:2007.09355 [cs]. – 2020. – pp. 1–21.
34. FakeInversion: Learning to Detect Images from Unseen Text-to-Image Models by Inverting Stable Diffusion. FakeInversion / G. Cazenavette [и др.] // arXiv:2406.08603 [cs]. – 2024. – pp. 1–40.
35. DIRE for Diffusion-Generated Image Detection / Z. Wang [и др.] // arXiv:2303.09295 [cs]. – 2023. – pp. 1–17.
36. Zhang Y. Diffusion Noise Feature: Accurate and Fast Generated Image Detection. Diffusion Noise Feature / Y. Zhang, X. Xu // arXiv:2312.02625 [cs]. – 2024. – pp. 1–23.
37. RenAIssance: A Survey into AI Text-to-Image Generation in the Era of Large Model. RenAIssance / F. Bie [и др.] // arXiv:2309.00810 [cs]. – 2023. – pp. 1–25.
38. DE-FAKE: Detection and Attribution of Fake Images Generated by Text-to-Image Generation Models. DE-FAKE / Z. Sha [и др.] // arXiv:2210.06998 [cs]. – 2023. – pp. 1–14.
39. BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation. BLIP / J. Li [и др.] // arXiv:2201.12086 [cs]. – 2022. – pp. 1–12.
40. ZeroFake: Zero-Shot Detection of Fake Images Generated and Edited by Text-to-Image Generation Models / Z. Sha [и др.] // Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security : CCS '24. – New York, NY, USA: Association for Computing Machinery, 2024. – ZeroFake. – pp. 4852–4866.
41. GenImage-Dataset [сайт] // GitHub. – URL: <https://github.com/GenImage-Dataset/GenImage> (дата обращения: 12.03.2025).
42. Bird J.J. CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images. CIFAKE / J.J. Bird, A. Lotfi // arXiv:2303.14126 [cs]. – 2023. – pp. 1–13.

43. ImageNet [сайт] // ImageNet. – URL: <https://www.image-net.org/index.php> (дата обращения: 12.03.2025).
44. tiny genimage [сайт] // Kaggle. – URL: <https://www.kaggle.com/datasets/yangsangtai/tiny-genimage> (дата обращения: 12.03.2025).
45. resnet50 – Torchvision main documentation [сайт] // PyTorch. – URL: <https://pytorch.org/vision/main/models/generated/torchvision.models.resnet50.html> (дата обращения: 13.03.2025).
46. Salesforce/blip-image-captioning-base · [сайт] // HuggingFace. – URL: <https://huggingface.co/Salesforce/blip-image-captioning-base> (дата обращения: 05.04.2025).
47. Kingma D.P. Adam: A Method for Stochastic Optimization. Adam / D.P. Kingma, J. Ba // arXiv:1412.6980 [cs]. – 2017. – pp. 1–15.
48. Lam K.F.Y. Confidence Intervals for the F1 Score: A Comparison of Four Methods. Confidence Intervals for the F1 Score / K.F.Y. Lam, V. Gopal, J. Qian // arXiv:2309.14621. – 2024. – pp. 1–31.
49. Casella G. Statistical inference / G. Casella, R.L. Berger. – 2. ed. – Pacific Grove, Calif: Duxbury, 2002. – 660 p.
50. CLIPScore: A Reference-free Evaluation Metric for Image Captioning. CLIPScore / J. Hessel [и др.] // arXiv:2104.08718 [cs]. – 2022. – pp. 1–15.

ПРИЛОЖЕНИЕ А

```
import torch.nn as nn
from torch import Tensor
from typing import Literal
import math
import torch, torchvision, diffusers
from typing import *
from
diffusers.pipelines.stable_diffusion.pipeline_stable_diffusion_i
mg2img import (
    retrieve_latents,
)

def get_resnet_model(mode: Literal["rgb", "ours"], norm_layer:
Literal["batch", "instance"], pretrained: bool) -> nn.Module:

    if norm_layer == "batch":
        norm = nn.BatchNorm2d
    elif norm_layer == "instance":
        norm = nn.InstanceNorm2d
    else:
        raise AssertionError("Unknown norm layer")

    if norm == nn.InstanceNorm2d:
        model = torchvision.models.resnet50(num_classes=1000,
pretrained=False, norm_layer=norm)

    model.load_state_dict(torchvision.models.ResNet50_Weights.IMAGEN
ET1K_V2.get_state_dict(progress=True, check_hash=True),
strict=False)
    else:
        model = torchvision.models.resnet50(num_classes=1000,
weights="IMAGENET1K_V2", norm_layer=norm)
```

```

    if mode == "ours":
        model.conv1.weight =
nn.Parameter(torch.cat([model.conv1.weight * 0.25] * 4, dim=1))
        model.conv1.in_channels = 12

    model.fc = nn.Linear(2048, 1)
    torch.nn.init.normal_(model.fc.weight.data, 0.0, 0.02)

    return model

def get_frq_resnet_model(mode: Literal["rgb", "ours", "frq"],
norm_layer: Literal["batch", "instance"], pretrained: bool) ->
nn.Module:

    if norm_layer == "batch":
        norm = nn.BatchNorm2d
    elif norm_layer == "instance":
        norm = nn.InstanceNorm2d
    else:
        raise AssertionError("Unknown norm layer")

    if norm == nn.InstanceNorm2d:
        model = torchvision.models.resnet50(num_classes=1000,
pretrained=False, norm_layer=norm)

    model.load_state_dict(torchvision.models.ResNet50_Weights.IMAGEN
ET1K_V2.get_state_dict(progress=True, check_hash=True),
strict=False)
    else:
        model = torchvision.models.resnet50(num_classes=1000,
weights="IMAGENET1K_V2", norm_layer=norm)

    if mode == "frq":

```

```

        model.conv1.weight =
nn.Parameter(torch.cat([model.conv1.weight * 0.25] * 2, dim=1))
        model.conv1.in_channels = 6

    model.fc = nn.Linear(2048, 1)
    torch.nn.init.normal_(model.fc.weight.data, 0.0, 0.02)

    return model

class ESPCN(nn.Module):
    def __init__(
        self,
        in_channels: int,
        out_channels: int,
        channels: int,
        upscale_factor: int,
    ) -> None:
        super(ESPCN, self).__init__()
        hidden_channels = channels // 2
        out_channels = int(out_channels * (upscale_factor ** 2))
        self.bn = nn.BatchNorm2d(in_channels)
        # Feature mapping
        self.feature_maps = nn.Sequential(
            nn.Conv2d(in_channels, channels, (5, 5), (1, 1), (2,
2)),
            nn.Tanh(),
            nn.Conv2d(channels, hidden_channels, (3, 3), (1, 1),
(1, 1)),
            nn.Tanh(),
        )

        # Sub-pixel convolution layer
        self.sub_pixel_0 = nn.Sequential(
            nn.Conv2d(hidden_channels, out_channels, (3, 3), (1,
1), (1, 1)),

```

```

        nn.PixelShuffle(upscale_factor),
        nn.Sigmoid(),
    )

    self.sub_pixel_1 = nn.Sequential(
        nn.Conv2d(hidden_channels, out_channels, (3, 3), (1,
1), (1, 1)),
        nn.PixelShuffle(upscale_factor),
        nn.Sigmoid(),
    )

    # Initial model weights
    for module in self.modules():
        if isinstance(module, nn.Conv2d):
            if module.in_channels == 32:
                nn.init.normal_(module.weight.data,
                                0.0,
                                0.001)
                nn.init.zeros_(module.bias.data)
            else:
                nn.init.normal_(module.weight.data,
                                0.0,
                                math.sqrt(2 /
(module.out_channels * module.weight.data[0][0].numel())))
                nn.init.zeros_(module.bias.data)

    def forward(self, x: Tensor) -> Tensor:
        return self._forward_impl(x)

    # Support torch.script function.
    def _forward_impl(self, x: Tensor) -> Tensor:
        x = self.bn(x)
        x = self.feature_maps(x)
        x_h = self.sub_pixel_0(x)
        x_l = self.sub_pixel_1(x)

```

```

        return x_h, x_l

def create_vae():
    # create SD pipeline as usual
    pipe =
diffusers.StableDiffusionPipeline.from_pretrained("runwayml/stable-diffusion-v1-5",

torch_dtype=torch.float16, safety_checker=None)
    vae = pipe.vae
    del pipe
    torch.cuda.empty_cache()

    return vae

class FIRE_model(nn.Module):
    def __init__(self, mode="frq", norm_layer="instance",
pretrained=True, radiuslow=40, radiushigh=120):
        super(FIRE_model, self).__init__()
        # initialize VAE in the DM
        self.vae = create_vae()
        self.decode_dtype =
next(iter(self.vae.post_quant_conv.parameters()))).dtype

        for param in self.vae.parameters():
            param.requires_grad = False

        # backend classifier
        self.resnet = get_frq_resnet_model(mode=mode,
norm_layer=norm_layer, pretrained=pretrained)
        # FMRE module
        self.fft_filter_module = fft_filter(radiuslow=radiuslow,
radiushigh=radiushigh, rows=256, cols=256)
        self.vae = self.vae.to("cuda")

```

```

        self.resnet = self.resnet.to("cuda")
        self.fft_filter_module =
self.fft_filter_module.to("cuda")

    def forward(self, x):
        middle_freq_image, middle_filtered_image, mask_mid_freq,
mask_mid_filterd = self.fft_filter_module(x)

        latents_x =
retrieve_latents(self.vae.encode(x.to(self.decode_dtype)))
        reconstructions_x = self.vae.decode(

latents_x.to(self.decode_dtype), return_dict=False
        )[0]

        latents_middle_filtered =
retrieve_latents(self.vae.encode(middle_filtered_image.to(self.d
ecode_dtype)))
        reconstructions_middle_filtered = self.vae.decode(

latents_middle_filtered.to(self.decode_dtype), return_dict=False
        )[0]

        raw_reconstructions_delta = torch.abs(reconstructions_x -
x)

        filtered_reconstructions_delta =
torch.abs(reconstructions_middle_filtered
middle_filtered_image)
        o = self.resnet(torch.cat([raw_reconstructions_delta,
filtered_reconstructions_delta], dim=1))

        return o, middle_freq_image, raw_reconstructions_delta,
mask_mid_freq, mask_mid_filterd

```

```

class fft_filter(nn.Module):
    def __init__(self, radiuslow=35, radiushigh=120, rows=256,
cols=256):
        super(fft_filter, self).__init__()
        self.radiuslow = radiuslow
        self.radiushigh = radiushigh
        self.rows = rows
        self.cols = cols
        i_mask, r_i_mask = self.init_mask()
        i_mask,          r_i_mask          =          nn.Parameter(i_mask,
requires_grad=False), nn.Parameter(r_i_mask, requires_grad=False)
        self.register_buffer('i_mask', i_mask)
        self.register_buffer('r_i_mask', r_i_mask)
        # # encode and decode frequency mask
        self.mask_autoencoder          =          ESPCN(in_channels=3,
out_channels=1, channels=64, upscale_factor=1)

    def init_mask(self):
        mask      =      torch.ones((1,      self.rows,      self.cols),
dtype=torch.float32, requires_grad=False)
        crow, ccol = self.rows // 2 , self.cols // 2
        center = [crow, ccol]
        x,      y      =      torch.meshgrid(torch.arange(self.rows),
torch.arange(self.cols), indexing='ij')
        mask_area = (x - center[0]) ** 2 + (y - center[1]) ** 2 <
self.radiuslow*self.radiuslow
        mask[:, mask_area] = 0
        mask_area = (x - center[0]) ** 2 + (y - center[1]) ** 2 >=
self.radiushigh*self.radiushigh
        mask[:, mask_area] = 0

        return mask, 1-mask

# implement FMRE

```

```

def middle_pass_filter(self, image):
    freq_image = torch.fft.fftn(image * 255, dim=(-2, -1))
    freq_image = torch.fft.fftshift(freq_image, dim=(-2, -1))
    mask_mid_frq,          mask_mid_filterd          =
self.mask_autoencoder((20*torch.log(torch.abs(freq_image)+1e-
7))/255)
    mask_mid_frq = mask_mid_frq.to(freq_image.dtype)

    middle_freq = freq_image * mask_mid_frq
    middle_freq = torch.fft.ifftshift(middle_freq, dim=(-2, -
1))

    masked_image_array = torch.fft.ifftn(middle_freq, dim=(-
2, -1))

    z = torch.abs(masked_image_array)
    _min = torch.min(z)
    _max = torch.max(z)
    z = (z)/(_max-_min)
    middle_freq_image = z

    mask_mid_filterd = mask_mid_filterd.to(freq_image.dtype)
    middle_filtered = freq_image * mask_mid_filterd
    middle_filtered = torch.fft.ifftshift(middle_filtered,
dim=(-2, -1))
    middle_filtered_array = torch.fft.ifftn(middle_filtered,
dim=(-2, -1))
    z = torch.abs(middle_filtered_array)
    _min = torch.min(z)
    _max = torch.max(z)
    z = (z)/(_max-_min)
    middle_filtered_image = z

    return      middle_freq_image,      middle_filtered_image,
mask_mid_frq.to(torch.float32),
mask_mid_filterd.to(torch.float32)

```



```
def forward(self, image):  
    middle_freq_image, middle_filtered_image, mask_mid_freq,  
mask_mid_filterd = self.middle_pass_filter(image)  
  
    return middle_freq_image, middle_filtered_image,  
mask_mid_freq, mask_mid_filterd
```

ПРИЛОЖЕНИЕ Б

```
import torch.nn as nn
from torch import Tensor
from typing import Literal
import math
import torch, torchvision, diffusers
from typing import *
from
diffusers.pipelines.stable_diffusion.pipeline_stable_diffusion_i
mg2img import (
    retrieve_latents,
)
from diffusers.schedulers import SchedulerMixin
from diffusers import PNDMScheduler

def get_resnet_model(mode: Literal["rgb", "ours"], norm_layer:
Literal["batch", "instance"], pretrained: bool) -> nn.Module:

    if norm_layer == "batch":
        norm = nn.BatchNorm2d
    elif norm_layer == "instance":
        norm = nn.InstanceNorm2d
    else:
        raise AssertionError("Unknown norm layer")

    if norm == nn.InstanceNorm2d:
        model = torchvision.models.resnet50(num_classes=1000,
pretrained=False, norm_layer=norm)

model.load_state_dict(torchvision.models.ResNet50_Weights.IMAGEN
ET1K_V2.get_state_dict(progress=True, check_hash=True),
strict=False)
    else:
```

```

        model = torchvision.models.resnet50(num_classes=1000,
weights="IMAGENET1K_V2", norm_layer=norm)

    if mode == "ours":
        model.conv1.weight =
nn.Parameter(torch.cat([model.conv1.weight * 0.25] * 4, dim=1))
        model.conv1.in_channels = 12

    model.fc = nn.Linear(2048, 1)
    torch.nn.init.normal_(model.fc.weight.data, 0.0, 0.02)

    return model

def get_frq_resnet_model(mode: Literal["rgb", "ours", "frq"],
norm_layer: Literal["batch", "instance"], pretrained: bool) ->
nn.Module:

    if norm_layer == "batch":
        norm = nn.BatchNorm2d
    elif norm_layer == "instance":
        norm = nn.InstanceNorm2d
    else:
        raise AssertionError("Unknown norm layer")

    if norm == nn.InstanceNorm2d:
        model = torchvision.models.resnet50(num_classes=1000,
pretrained=False, norm_layer=norm)

    model.load_state_dict(torchvision.models.ResNet50_Weights.IMAGEN
ET1K_V2.get_state_dict(progress=True, check_hash=True),
strict=False)
    else:
        model = torchvision.models.resnet50(num_classes=1000,
weights="IMAGENET1K_V2", norm_layer=norm)

```

```

    if mode == "freq":
        model.conv1.weight =
nn.Parameter(torch.cat([model.conv1.weight * 0.25] * 2, dim=1))
        model.conv1.in_channels = 6

    model.fc = nn.Linear(2048, 1)
    torch.nn.init.normal_(model.fc.weight.data, 0.0, 0.02)

    return model

class ESPCN(nn.Module):
    def __init__(
        self,
        in_channels: int,
        out_channels: int,
        channels: int,
        upscale_factor: int,
    ) -> None:
        super(ESPCN, self).__init__()
        hidden_channels = channels // 2
        out_channels = int(out_channels * (upscale_factor ** 2))
        self.bn = nn.BatchNorm2d(in_channels)
        # Feature mapping
        self.feature_maps = nn.Sequential(
            nn.Conv2d(in_channels, channels, (5, 5), (1, 1), (2,
2)),
            nn.Tanh(),
            nn.Conv2d(channels, hidden_channels, (3, 3), (1, 1),
(1, 1)),
            nn.Tanh(),
        )

        # Sub-pixel convolution layer
        self.sub_pixel_0 = nn.Sequential(

```

```

        nn.Conv2d(hidden_channels, out_channels, (3, 3), (1,
1), (1, 1)),
        nn.PixelShuffle(upscale_factor),
        nn.Sigmoid(),
    )

    self.sub_pixel_1 = nn.Sequential(
        nn.Conv2d(hidden_channels, out_channels, (3, 3), (1,
1), (1, 1)),
        nn.PixelShuffle(upscale_factor),
        nn.Sigmoid(),
    )

    # Initial model weights
    for module in self.modules():
        if isinstance(module, nn.Conv2d):
            if module.in_channels == 32:
                nn.init.normal_(module.weight.data,
                                0.0,
                                0.001)
                nn.init.zeros_(module.bias.data)
            else:
                nn.init.normal_(module.weight.data,
                                0.0,
                                math.sqrt(2
(module.out_channels * module.weight.data[0][0].numel()))
                nn.init.zeros_(module.bias.data)

    def forward(self, x: Tensor) -> Tensor:
        return self._forward_impl(x)

    # Support torch.script function.
    def _forward_impl(self, x: Tensor) -> Tensor:
        x = self.bn(x)
        x = self.feature_maps(x)

```

```

        x_h = self.sub_pixel_0(x)
        x_l = self.sub_pixel_1(x)

        return x_h, x_l

def create_pipeline():
    # create SD pipeline as usual
    pipe =
diffusers.StableDiffusionPipeline.from_pretrained("runwayml/stable-diffusion-v1-5",
torch_dtype=torch.float16, safety_checker=None)
    vae = pipe.vae
    text_encoder = pipe.text_encoder
    tokenizer = pipe.tokenizer
    unet = pipe.unet

    scheduler_config = pipe.scheduler.config
    scheduler1 = pipe.scheduler
    scheduler2 = PNDMScheduler.from_config(scheduler_config)

    del pipe

    torch.cuda.empty_cache()

    return vae, text_encoder, tokenizer, unet, scheduler1, scheduler2

class FIRE_BLIP_model(nn.Module):
    def __init__(self, mode="frq", norm_layer="instance", pretrained=True, radiuslow=40, radiushigh=120):
        super(FIRE_BLIP_model, self).__init__()
        # initialize VAE in the DM
        self.vae, self.text_encoder, self.tokenizer, self.unet, self.scheduler1, self.scheduler2 = create_pipeline()

```

```

        self.decode_dtype =
next(iter(self.vae.post_quant_conv.parameters()))).dtype

    for param in self.vae.parameters():
        param.requires_grad = False

    for param in self.text_encoder.parameters():
        param.requires_grad = False

    for param in self.unet.parameters():
        param.requires_grad = False

    # backend classifier
    self.resnet = get_frq_resnet_model(mode=mode,
norm_layer=norm_layer, pretrained=pretrained)
    # FMRE module
    self.fft_filter_module = fft_filter(radiuslow=radiuslow,
radiushigh=radiushigh, rows=IMAGE_SIZE, cols=256)
    self.vae = self.vae.to("cuda")

    self.text_encoder = self.text_encoder.to("cuda")
    self.unet = self.unet.to("cuda")

self.scheduler1.set_timesteps(num_inference_steps=NUM_INFERENCE_
STEPS)

self.scheduler2.set_timesteps(num_inference_steps=NUM_INFERENCE_
STEPS)

    self.resnet = self.resnet.to("cuda")
    self.fft_filter_module
self.fft_filter_module.to("cuda")

def forward(self, x, blip_label):

```

```

        middle_freq_image, middle_filtered_image, mask_mid_freq,
mask_mid_filterd = self.fft_filter_module(x)

        latents_x =
retrieve_latents(self.vae.encode(x.to(self.decode_dtype)))

        text_input = self.tokenizer(blip_label,
padding="max_length", truncation=True,
max_length=self.tokenizer.model_max_length,
return_tensors="pt").input_ids
        text_input = text_input.to("cuda").requires_grad_(False)
        text_embedding =
self.text_encoder(text_input)[0].requires_grad_(False)

        noise = torch.randn_like(latents_x).to("cuda")
        noisy_latents_x = self.scheduler1.add_noise(latents_x,
noise, timesteps=torch.tensor(TIMESTEPS))
        noise_pred = self.unet(noisy_latents_x,
torch.tensor(TIMESTEPS),
encoder_hidden_states=text_embedding).sample
        latents_x_new =
self.scheduler1._get_prev_sample(noisy_latents_x,
torch.tensor(TIMESTEPS), torch.tensor(TIMESTEPS - 10),
noise_pred)

        reconstructions_x =
self.vae.decode(latents_x_new.to(self.decode_dtype), return_dict=
False)[0]
        # reconstructions_x =
self.vae.decode(latents_x.to(self.decode_dtype), return_dict=False)
[0]

        latents_middle_filtered =
retrieve_latents(self.vae.encode(middle_filtered_image.to(self.d
ecode_dtype)))

```



```

        noise_filtered =
torch.randn_like(latents_middle_filtered).to("cuda")
        noisy_latents_filtered =
self.scheduler2.add_noise(latents_middle_filtered,
noise_filtered, timesteps=torch.tensor(TIMESTEPS))
        noise_pred_filtered = self.unet(noisy_latents_filtered,
torch.tensor(TIMESTEPS),
encoder_hidden_states=text_embedding).sample
        latents_middle_filtered_new=
self.scheduler2._get_prev_sample(noisy_latents_filtered,
torch.tensor(TIMESTEPS), torch.tensor(TIMESTEPS - 10),
noise_pred_filtered)

        reconstructions_middle_filtered =
self.vae.decode(latents_middle_filtered_new.to(self.decode_dtype
), return_dict=False)[0]
        filtered_reconstructions_delta =
torch.abs(reconstructions_middle_filtered -
middle_filtered_image)
        o = self.resnet(torch.cat([raw_reconstructions_delta,
filtered_reconstructions_delta], dim=1))

        return o, middle_freq_image, raw_reconstructions_delta,
mask_mid_freq, mask_mid_filtered

```

```

class fft_filter(nn.Module):
    def __init__(self, radiuslow=35, radiushigh=120,
rows=IMAGE_SIZE, cols=IMAGE_SIZE):
        super(fft_filter, self).__init__()
        self.radiuslow = radiuslow
        self.radiushigh = radiushigh
        self.rows = rows
        self.cols = cols

```

```

        # preset masks M_{mid} and M_{mid_c}
        i_mask, r_i_mask = self.init_mask()
        i_mask, r_i_mask = nn.Parameter(i_mask,
requires_grad=False), nn.Parameter(r_i_mask, requires_grad=False)
        self.register_buffer('i_mask', i_mask)
        self.register_buffer('r_i_mask', r_i_mask)
        # # encode and decode frequency mask
        self.mask_autoencoder = ESPCN(in_channels=3,
out_channels=1, channels=64, upscale_factor=1)

    def init_mask(self):
        mask = torch.ones((1, self.rows, self.cols),
dtype=torch.float32, requires_grad=False)
        crow, ccol = self.rows // 2, self.cols // 2
        center = [crow, ccol]
        x, y = torch.meshgrid(torch.arange(self.rows),
torch.arange(self.cols), indexing='ij')
        mask_area = (x - center[0]) ** 2 + (y - center[1]) ** 2 <
self.radiuslow*self.radiuslow
        mask[:, mask_area] = 0
        mask_area = (x - center[0]) ** 2 + (y - center[1]) ** 2 >=
self.radiushigh*self.radiushigh
        mask[:, mask_area] = 0

        return mask, 1-mask

    # implement FMRE
    def middle_pass_filter(self, image):
        freq_image = torch.fft.fftn(image * 255, dim=(-2, -1))
        freq_image = torch.fft.fftshift(freq_image, dim=(-2, -1))
        mask_mid_frq, mask_mid_filterd =
self.mask_autoencoder((20*torch.log(torch.abs(freq_image))+1e-
7))/255)
        mask_mid_frq = mask_mid_frq.to(freq_image.dtype)

```

```

        middle_freq = freq_image * mask_mid_frq
        middle_freq = torch.fft.ifftshift(middle_freq, dim=(-2, -
1))

        masked_image_array = torch.fft.ifftn(middle_freq, dim=(-
2, -1))

        z = torch.abs(masked_image_array)
        _min = torch.min(z)
        _max = torch.max(z)
        z = (z)/(_max-_min)
        middle_freq_image = z

        mask_mid_filterd = mask_mid_filterd.to(freq_image.dtype)
        middle_filtered = freq_image * mask_mid_filterd
        middle_filtered = torch.fft.ifftshift(middle_filtered,
dim=(-2, -1))

        middle_filtered_array = torch.fft.ifftn(middle_filtered,
dim=(-2, -1))

        z = torch.abs(middle_filtered_array)
        _min = torch.min(z)
        _max = torch.max(z)
        z = (z)/(_max-_min)
        middle_filtered_image = z

        return middle_freq_image, middle_filtered_image,
mask_mid_frq.to(torch.float32),
mask_mid_filterd.to(torch.float32)

    def forward(self, image):
        middle_freq_image, middle_filtered_image, mask_mid_frq,
mask_mid_filterd = self.middle_pass_filter(image)

        return middle_freq_image, middle_filtered_image,
mask_mid_frq, mask_mid_filterd

```